

A Highly Efficient Numerically Exact Algorithm for Two-Dimensional Bin-Packing Problems

Sunkanghong Wang^a, Roberto Baldacci^b, Fabio Furini^c, Lijun Wei^a, Qiang Liu^a

^aGuangdong Provincial Key Laboratory of Computer Integrated Manufacturing System, School of Electromechanical Engineering, Guangdong University of Technology, 510006 Guangzhou, Guangdong, China

^bCollege of Science and Engineering, Hamad Bin Khalifa University, Qatar Foundation, Doha, Qatar

^cDepartment of Computer, Control, and Management Engineering “Antonio Ruberti”, Sapienza University of Rome, 00185 Rome, Italy

Contact: ✉ wskh0929@gmail.com (Sunkanghong Wang), ✉ rbaldacci@hbku.edu.qa (Roberto Baldacci), ✉ fabio.furini@uniroma1.it (Fabio Furini), ✉ villagerwei@gdut.edu.cn (Lijun Wei), ✉ liuqiang@gdut.edu.cn (Qiang Liu)

Abstract. This paper explores the classic Two-Dimensional Bin-Packing Problem and its most famous variant with Guillotine Constraints, both of which require packing rectangular items into the fewest rectangular bins and have been extensively studied for decades because of their wide-life applications and strong $\mathcal{NP}$ -hardness. Unlike previous work, we propose the first unified numerically exact algorithm for solving both problems that not only addresses numerical-safety issues but also demonstrates excellent computational performance by effectively integrating several novel, well-designed primal heuristics and advanced techniques. Comprehensive computational experiments reveal that the proposed exact and heuristic algorithms substantially outperform state-of-the-art methods, solve dozens of instances that had remained unsolved for decades, and use fewer bins while significantly reducing computation time. For the two problems, nearly all classic instances and most newly generated large-scale instances with up to 500 items can be solved optimally within one hour, whereas the remaining instances have very small optimality gaps. These results constitute a significant advancement in two-dimensional bin packing and underscore the potential of the proposed approach for tackling higher-dimensional and related problems. To support future research, we provide a comprehensive repository containing the algorithm’s code accompanied by detailed documentation.

Funding: This study was supported in part by the Natural Science Foundation of China under Grant 72271062 and Grant 52575565, in part by the Science Fund for Distinguished Young Scholars of Guangdong Province under Grant 2022B1515020076, in part by the Science and Technology Projects in Guangzhou under Grant 2024A04J01019, and in part by the Province Natural Science Fund of Guangdong under Grant 2025A1515011297 and Grant 2024A1515010246.

Key words: packing and cutting • two-dimensional bin-packing • guillotine cuts • exact algorithm • numerical precision

History: September 21, 2025

1. Introduction

In this paper, we investigate the well-known Two-Dimensional Bin-Packing Problem (2BPP) and its most famous variant with guillotine constraints (2BPP-G). Both problems belong to one of the oldest areas of combinatorial optimization—*packing and cutting*—where the objective is to pack items into larger spaces or cut raw materials into smaller pieces to maximize profit or minimize waste.

Formally, we are given an infinite number of identical rectangular bins, each with width $W \in \mathbb{Z}^+$ and height $H \in \mathbb{Z}^+$, and a set $\mathcal{I} = \{1, \dots, n\}$ of n rectangular items, where each item $i \in \mathcal{I}$ has width $w_i \in \mathbb{Z}^+$ and height $h_i \in \mathbb{Z}^+$. The 2BPP asks to determine the minimum number of bins to pack all the items without overlap or rotation, with item edges parallel to the bin edges; this placement is known as *orthogonal packing*.

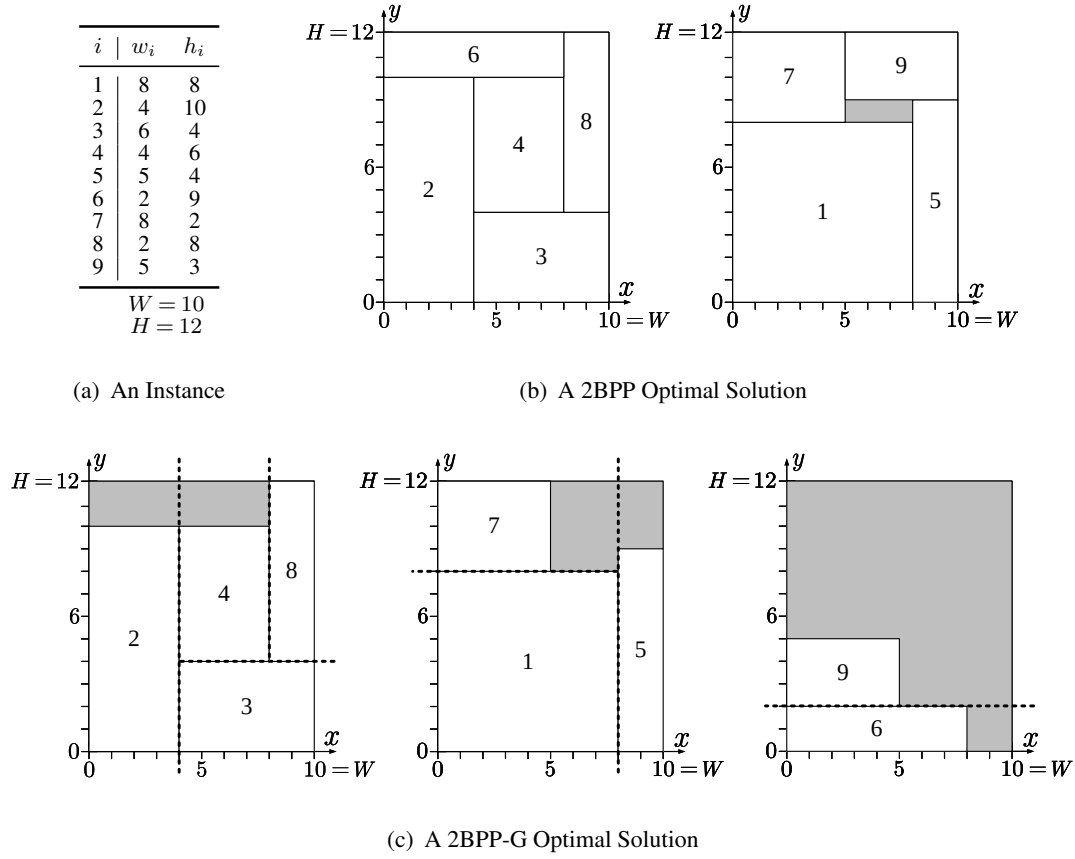


Figure 1 Illustrations of the Optimal 2BPP and 2BPP-G Solutions for a Simple Instance

On this basis, the 2BPP-G additionally requires that items must be obtained from the bins through a series of edge-to-edge cuts parallel to the bin edges. These cuts are referred to as *guillotine cuts*.

The table in Figure 1(a) provides a simple 2BPP(-G) instance and lists the corresponding widths and heights of the $n = 9$ items along with the bin dimensions. Figure 1(b) illustrates one optimal 2BPP solution for this instance, in which the items are orthogonally packed into two bins. The first bin is completely filled, whereas the second bin contains a small unused region at its center, shown shaded in grey. Figure 1(c) depicts one optimal 2BPP-G solution for the same instance; however, the items cannot be orthogonally packed into two bins due to the guillotine constraints, and the bold dashed lines indicate the guillotine cuts.

Due to the fundamental mathematical properties of rectangles, both 2BPP and 2BPP-G play a critical role in numerous real-world applications across various fields. A typical example is the cutting of raw materials (e.g., paper and metal) in *manufacturing* (Firat and Alpaslan 2020), where 2BPP-G is particularly relevant to the high-demand glass and wood industries, as guillotine cutting is faster and has become one of the most common operational constraints (Furini et al. 2016). Other important examples include the optimization of software fault injection in *computer systems* (Salih and Gopal 2024), the loading of items that cannot be stacked to avoid damage in *logistics* (Iori et al. 2007, Zhang et al. 2022b), the allocation of berths in *maritime*

Table 1 Summary of Abbreviations and Their Meanings

Abbreviation	Meaning	Abbreviation	Meaning
2BPP	Two-Dimensional Bin-Packing Problem	2BPP-G	2BPP with Guillotine Constraints
BPP	Bin-Packing Problem	BB	Branch-and-Bound
BP	Branch-and-Price	BPC	Branch-Price-and-Cut
LP	Linear Programming	MILP	Mixed Integer Linear Programming
SR	Subset-Row	VB	Valid-Bound
MP	Master Problem	CRG	Column-and-Row Generation
LMP	LP relaxation of MP	RF	Ryan-Foster
RLMP	restricted LMP	PP	Pricing Problem

(Bierwirth and Meisel 2010, Xu and Lee 2018), and the allocation of downlink zones in *telecommunications* (Lodi et al. 2011, Martello 2014). From a computational standpoint, these two problems generalize the classic, strongly $\mathcal{NP}$ -hard Bin Packing Problem (BPP) (Garey and Johnson 1979). They pose significant challenges and practical value, and have therefore attracted extensive research attention for decades.

In the following subsections, we first review the related progress and provide our research motivation, then present the major contributions and organization of this paper. To enhance readability, the abbreviations used and their meanings are summarized in Table 1, arranged in the order of their introduction.

1.1. Related Work and Motivation

2BPP(-G). Since the pioneering work of Gilmore and Gomory (1965), which first addressed 2BPP, the literature has explored numerous approaches, including heuristic methods, bounding techniques, and exact algorithms. Table 2 summarizes the contributions of classic and advanced 2BPP(-G) studies. We refer interested readers to the surveys by Lodi et al. (2002b), Iori et al. (2021), and Francescatto and Júnior (2025) for more comprehensive reviews of solution approaches and practical applications, as well as to the recent work of Liu et al. (2024) for new insights into the performance of 2BPP mathematical models (Chen et al. 1995, Pisinger and Sigurd 2007, Castro and Oliveira 2011, Blum et al. 2012).

Four exact algorithms have been proposed for the 2BPP. The first is a branch-and-bound (BB) algorithm proposed by Martello and Vigo (1998). It adopts a nested branching scheme: each node of the outer tree assigns an item to a bin, and the inner tree checks whether all assigned items can be packed into that bin using bottom-left placement. Clautiaux et al. (2007) later designed a similar BB algorithm that solves the two smaller problems and embeds new lower bounds (Carlier et al. 2007). The two most recent advances rely on model decomposition. Pisinger and Sigurd (2007) introduced a branch-and-price (BP) algorithm based on the classic set-covering formulation and solved the pricing problem as a constraint-satisfaction problem within a branch-and-cut framework. Côté et al. (2021) proposed a logic-based Benders decomposition that integrates advanced preprocessing procedures, valid inequalities, and bounding techniques. Several heuristic algorithms have also been developed, including the tabu search approach proposed by Lodi et al. (1999), the guided local search approach proposed by Faroe et al. (2003), the hybrid GRASP/VND approach proposed

Table 2 Methodological Contributions of Classic and Advanced 2BPP(-G) Studies

Problem	Short name - Reference	Methodological contributions					Exact	N-Exact
		Heuristics	Lower bounds	BD	BB	BP		
2BPP	BW87 - Berkey and Wang (1987)	✓						
	MV98 - Martello and Vigo (1998)		✓		✓		✓	✓
	BM03 - Boschetti and Mingozzi (2003a,b)	✓	✓					
	MT06 - Monaci and Toth (2006)	✓						
	CaCIM07 - Carlier et al. (2007)		✓					
	ClCaM07 - Clautiaux et al. (2007)				✓		✓	✓
	PS07 - Pisinger and Sigurd (2007)					✓	✓	
	PAOT10 - Parreño et al. (2010)	✓						
	WOZL13 - Wei et al. (2013)	✓						
2BPP-G	CHIP21/CHIB21 - Côté et al. (2021)			✓			✓	
	PS07 - Pisinger and Sigurd (2007)					✓	✓	
	CHBP11 - Charalambous and Fleszar (2011)	✓						
	CFIH13 - Fleszar (2013)	✓						
	HHA14 - Hong et al. (2014)	✓						
	ENH17 - Lodi et al. (2017)	✓						
	HHTB18 - Cui et al. (2018)	✓						
	GDRR22 - Gardeyn and Wauters (2022)	✓						

Notes. BD: Benders decomposition; BB: branch-and-bound; BP: branch-and-price; N-Exact: Numerically Exact.

by [Parreño et al. \(2010\)](#), the goal-driven approach proposed by [Wei et al. \(2013\)](#), and other methods based on greedy heuristics ([Boschetti and Mingozzi 2003b](#), [Monaci and Toth 2006](#), [Zhang et al. 2011](#)).

For the 2BPP-G, to the best of our knowledge, the BP algorithm of [Pisinger and Sigurd \(2007\)](#) remains the only exact method in the literature. This algorithm is adapted from their BP for the 2BPP by verifying at each step of the pricing procedure that the current placement satisfies the guillotine constraints. Most subsequent studies address the problem heuristically, including the constructive heuristic of [Charalambous and Fleszar \(2011\)](#), the critical fit insertion method of [Fleszar \(2013\)](#), the hybrid heuristic of [Hong et al. \(2014\)](#), the partial enumeration heuristic of [Lodi et al. \(2017\)](#), the hybrid heuristic with triple block patterns of [Cui et al. \(2018\)](#), and the goal-driven ruin-and-recreate method of [Gardeyn and Wauters \(2022\)](#).

According to the above brief review, the following three major issues can be identified. First, despite the close connection between 2BPP and 2BPP-G—for example, the former is a relaxation of the latter and usually very tight (as analyzed in §EC.2.4 of our E-Companion)—almost all works treat them separately. Second, although a large number of heuristic methods have been developed, only a few studies have explored exact algorithms due to their high computational complexity. In particular, there has been no new progress in the exact 2BPP-G algorithm for almost twenty years, while many effective techniques have been developed recently for related problems (see the following sections). Third, the classic benchmark set established by [Berkey and Wang \(1987\)](#) and [Martello and Vigo \(1998\)](#) has been widely used to evaluate 2BPP(-G) algorithms for a long time, comprising 500 instances with up to 100 items. However, dozens of instances remain unsolved optimally, including some with as few as 40 items. These issues motivate the development of a new efficient exact algorithm for solving 2BPP and 2BPP-G uniformly in this paper.

BPP. In contrast, following the pattern-based and arc-flow-based solution paradigms pioneered by Gilmore and Gomory (1961) and Valério de Carvalho (1999), exact methods for BPP have been extensively studied, with leading algorithms able to solve instances with thousands of items optimally (see, e.g., Delorme and Iori (2020), Wei et al. (2020b), de Lima et al. (2023), da Silva and Schouery (2023), Baldacci et al. (2024)), where the pattern-based branch-price-and-cut (BPC) algorithm by da Silva and Schouery (2023) performs best on average on the benchmark instances. This approach reformulates BPP as the set-covering model with exponentially many variables and solves it exactly via column generation and BB. Due to the similarity between BPP and 2BPP(-G), adapting it to solve 2BPP(-G) is a promising choice.

However, the adaptation presents significant challenges because searching for the best two-dimensional packing pattern is usually strongly $\mathcal{NP}$ -hard. This leads to an extremely difficult pricing problem, which greatly increases the computational overhead of column generation at each node. To address this, Pisinger and Sigurd (2007) carefully designed an exact pricing algorithm. Unfortunately, the resulting BP algorithm can still only solve 435 and 377 of the 500 classic benchmark instances for 2BPP and 2BPP-G, respectively. Our experiment in §5.3 also shows that even with state-of-the-art exact techniques for solving the pricing problem, column generation at the root node cannot be completed within an hour on a large number of instances. Therefore, it is essential to explore effective heuristic subroutines and theoretically guaranteed acceleration strategies to overcome these difficulties and improve the exact solution process.

Numerical Safety Issues. On the other hand, traditional pricing algorithms typically operate on the optimal dual solution obtained by floating-point-based Linear Programming (LP) solvers (such as CPLEX and GUROBI). However, these dual solutions may be suboptimal or infeasible due to *cancellation errors*, and such solvers cannot provide a bound on their accuracy (Held et al. 2012). Several techniques have been proposed to recover a valid dual bound from a suboptimal or infeasible dual solution (Farley 1990, Lübbecke and Desrosiers 2005); these require pricing algorithms to be *error-free*. If not handled carefully, cancellation errors in pricing may cause two severe numerical issues: (i) column generation may enter an infinite loop because pricing algorithms may repeatedly generate patterns with negative reduced cost that lie within the solver tolerance but cannot enter the basis, and (ii) column generation may terminate incorrectly because invalid bounds are computed. The first issue can be resolved by explicitly prohibiting the generation of any patterns already added in the master problem, while the second issue can be addressed by avoiding uncertain cancellation errors through integer-based computations (see, e.g., Pessoa et al. (2020), da Silva and Schouery (2023), and Baldacci et al. (2024)). More generally, any Mixed Integer Linear Programming (MILP) procedure that relies on a floating-point solver to compute bounds may result in numerical inaccuracies. A more comprehensive review of numerically safe methods can be found in Baldacci et al. (2024).

Despite these advances, in the context of 2BPP(-G), only the two early exact algorithms proposed by Martello and Vigo (1998) and Clautiaux et al. (2007) can be considered numerically safe, as they rely

on floating-point-free computations (recall Table 2). Nevertheless, these algorithms are not capable of efficiently solving hundreds of benchmark instances. In contrast, the more advanced exact algorithms by Pisinger and Sigurd (2007) and Côté et al. (2021), while computationally more powerful, cannot be considered numerically safe and may yield incorrect results, as they rely on floating-point LP solvers. The same holds for the MILP approaches (Chen et al. 1995, Castro and Oliveira 2011, Blum et al. 2012), which are generally limited to solving only small-sized instances and cannot be considered numerically safe. This motivates the development of an efficient and numerically exact 2BPP(-G) algorithm in this paper.

1.2. Contributions and Organization of This Paper

We believe that this paper stands out for its methodological, computational, and applied contributions:

- We introduce the first unified numerically exact BPC algorithm for solving both 2BPP and 2BPP-G. The algorithm incorporates state-of-the-art techniques from related approaches for packing and routing problems, tailored here for the first time to these two problems. It introduces new, effective methods to accelerate lower bound computation through a combined column-and-row-generation scheme and employs several original primal heuristics designed to complement and enhance the exact algorithm. Unlike previous algorithms, it also integrates numerical techniques to ensure safe computation without sacrificing efficiency.
- These innovations enable the proposed exact algorithm to solve nearly all classic instances without numerical inaccuracies, some of which have remained open for over three decades, establishing new benchmark results and making a notable computational contribution. Additionally, it demonstrates strong competitiveness with existing algorithms and successfully handles large-scale instances with up to 500 items—a significant advance in the field. The proposed hybrid heuristic is also highly competitive with leading alternative methods for 2BPP(-G) in the literature, finding higher-quality solutions in less time.
- Finally, although this paper focuses on 2BPP(-G), we believe that most of the ideas presented can be easily transferred to other related variants to address broader practical needs, thanks to their fundamental nature. Thus, the paper provides a comprehensive repository for these long-term research problems, including the algorithm’s code, benchmark instances, aggregated results, and solutions, thereby greatly supporting future research, comparison of solution approaches, and especially the numerous real-world applications in *manufacturing, computer systems, logistics, maritime, and telecommunications*, as discussed earlier.

The remainder of this paper is organized as follows. Section 2 outlines the proposed numerically exact BPC algorithm for solving the 2BPP(-G) uniformly. Sections 3 and 4 focus on describing the algorithm details. Section 5 presents and analyzes the computational results of our algorithm and compares them with those of state-of-the-art algorithms. Finally, Section 6 concludes the paper and suggests interesting future research directions. Additional material is provided in the E-Companion of the paper.

2. Numerically Exact Branch-Price-and-Cut Algorithm

Before proceeding, we refer readers to Barnhart et al. (1998), Lübbecke and Desrosiers (2005) for a comprehensive overview of column generation, to the recent books by Desrosiers et al. (2024) and Uchoa et al. (2024) for an up-to-date and thorough review of BPC algorithms and related techniques, and to Baldacci et al. (2024), Wang et al. (2025a) for recent advancements in numerically exact algorithms.

Hereafter, we denote the horizontal and vertical coordinates of the bottom-left corner of an item $i \in \mathcal{I}$ by (x_i, y_i) , where $x_i \in \mathbb{N} \cap [0, W - w_i]$ and $y_i \in \mathbb{N} \cap [0, H - h_i]$. These coordinates are restricted to integer values because both item and bin dimensions are integers (see, e.g., Iori et al. 2021, Côté et al. 2021). Without loss of generality, we assume that $\mathcal{I}$ is sorted in non-increasing order of item areas $\{w_i h_i\}_{i \in \mathcal{I}}$. Each bin is located in the positive orthant of the Cartesian plane with its bottom-left corner at $(0, 0)$.

2.1. Master Problem: A Unified Pattern-Based Formulation

For the 2BPP, a *feasible packing pattern* $\mathcal{I}_p \subseteq \mathcal{I}$, referred to simply as *pattern* p , is a set of non-overlapping items in a bin, where each item $i \in \mathcal{I}_p$ is orthogonally packed at a specific position (x_i, y_i) . Formally, for any pair of items i and j in $\mathcal{I}_p$ with $i < j$, at least one of the following inequalities holds: $x_i + w_i \leq x_j$, $x_j + w_j \leq x_i$, $y_i \geq y_j + h_j$, $y_j \geq y_i + h_i$. These inequalities ensure that item i is to the left, right, above, or below item j and therefore prevent overlap. Equivalently, a subset of (packed) items is a pattern if

$$[x_i, x_i + w_i) \cap [x_j, x_j + w_j) = \emptyset \quad \text{or} \quad [y_i, y_i + h_i) \cap [y_j, y_j + h_j) = \emptyset, \quad \forall i, j \in \mathcal{I}_p, i < j, \quad (1)$$

so that the horizontal or vertical intervals occupied by any two items do not intersect.

Additionally, for 2BPP-G, we require that the pattern p is *guillotine-cuttable* (i.e., it satisfies the guillotine constraints), meaning that it must meet at least one of the following conditions: (i) it contains at most three items; or (ii) there exists $x \in (0, W)$ or $y \in (0, H)$ such that p can be separated into two nonempty disjoint subpatterns, p_1 and p_2 , such that either $\mathcal{I}_{p_1} = \{i \in \mathcal{I}_p : x_i + w_i \leq x\}$ and $\mathcal{I}_{p_2} = \{i \in \mathcal{I}_p : x_i \geq x\}$, or $\mathcal{I}_{p_1} = \{i \in \mathcal{I}_p : y_i + h_i \leq y\}$ and $\mathcal{I}_{p_2} = \{i \in \mathcal{I}_p : y_i \geq y\}$. Both subpatterns must also be guillotine-cuttable.

Let $\mathcal{P}$ denote the set of all orthogonal-packing patterns when solving 2BPP or guillotine-cuttable patterns when solving 2BPP-G. For each item $i \in \mathcal{I}$ and pattern $p \in \mathcal{P}$, define the coefficient a_{ip} as 1 if $i \in \mathcal{I}_p$ and 0 otherwise. For each pattern $p \in \mathcal{P}$, introduce the binary variable ξ_p , which equals 1 when pattern p is selected in a solution and 0 otherwise. The following classic set-covering formulation, which contains an exponential number of variables, models the 2BPP and 2BPP-G uniformly:

$$\min \quad \sum_{p \in \mathcal{P}} \xi_p, \quad (2a)$$

$$\text{s. t.} \quad \sum_{p \in \mathcal{P}} a_{ip} \xi_p \geq 1, \quad \forall i \in \mathcal{I}, \quad (2b)$$

$$\xi_p \in \{0, 1\}, \quad \forall p \in \mathcal{P}. \quad (2c)$$

The objective function (2a) minimizes the number of selected patterns, which equals the number of bins used. Constraints (2b) guarantee that every item $i \in \mathcal{I}$ is included in at least one selected pattern.

To strengthen the LP relaxation of the model (2), we introduce the following two families of inequalities. Let $\mathcal{T}$ denote the index set of all item triplets and, for each $t \in \mathcal{T}$, let $\mathcal{T}_t \subseteq \mathcal{I}$ be the corresponding triplet of items. Below, we refer to an index t simply as a triplet. For each triplet $t \in \mathcal{T}$, let $\mathcal{P}(t) \subseteq \mathcal{P}$ be the subset of patterns that contain at least two items of the triplet, that is, $\mathcal{P}(t) = \{p \in \mathcal{P} : |\mathcal{I}_p \cap \mathcal{T}_t| \geq 2\}$. The first family of inequalities is as follows:

$$\sum_{p \in \mathcal{P}(t)} \xi_p \leq 1, \quad \forall t \in \mathcal{T}. \quad (3)$$

Constraints (3) impose that, for each triplet, at most one variable associated with patterns containing at least two items of the triplet can be selected. They form a subset of the *subset-rows* (SR) cuts of cardinality three introduced by Jepsen et al. (2008) for the Vehicle Routing Problem with Time Windows. Such cuts are particularly effective in column-generation approaches for routing, packing, and cutting problems (Baldacci et al. 2011, Contardo and Martinelli 2014, Pecin et al. 2017, Wei et al. 2020b, da Silva and Schouery 2023, Baldacci et al. 2024, Wang et al. 2025d). In addition, let $\widetilde{LB}$ denote a valid lower bound on the value of the objective function (2a). Accordingly, a second inequality, valid for model (2), reads as follows:

$$\sum_{p \in \mathcal{P}} \xi_p \geq \widetilde{LB}. \quad (4)$$

The constraint (4) forces the objective value of model (2) to be no less than the lower bound $\widetilde{LB}$. This *valid-bound* (VB) cut has been used effectively in cardinality optimization problems (Furini and Malaguti 2013, Serairi and Haouari 2018, Wei et al. 2020a, Wang et al. 2025d). The basic model (2), strengthened with inequalities (3) and (4), serves as the so-called *Master Problem* (MP) of the proposed BPC algorithm.

2.2. Algorithm Overview

This section outlines the BPC algorithm, covering preprocessing, computation of initial bounds, solution of the root node, and the branching and search strategies that guide the exploration of the solution space.

Preprocessing. The algorithm starts by deciding whether to rotate the instance. Let $\mathcal{I}_w = \{i \in \mathcal{I} : w_i > h_i \text{ and } w_i \leq W/2 \text{ and } h_i \leq H/2\}$ and $\mathcal{I}_h = \{i \in \mathcal{I} : h_i > w_i \text{ and } w_i \leq W/2 \text{ and } h_i \leq H/2\}$ be the subsets of items where the width is greater than the height, or the height is greater than the width, respectively, with both dimensions not exceeding half of the bin's width and height. If $|\mathcal{I}_w| \geq |\mathcal{I}_h|$ or $\sum_{i \in \mathcal{I}} (w_i - h_i) \geq 0$, the widths and heights of the bins and items are swapped. This strategy is adopted based on the observation that the heuristics used to compute primal solutions within the BPC algorithm are more effective on instances in which a greater proportion of items are taller than they are wide. Note that items with a width or height greater than $\frac{W}{2}$ or $\frac{H}{2}$ are excluded, as they are typically placed against one edge of the bin and thus have little impact on the algorithm. Related experiments can be found in §5.3 of the E-Companion. To reduce the instance size, we employ a *bin-item resizing* procedure, a widely adopted technique in cutting and packing problems due to its broad applicability (see, e.g., Boschetti et al. (2002), Carlier et al. (2007), Alvarez-Valdés et al. (2009), Côté et al. (2021)). Specifically, the procedure involves iteratively shrinking the bin

and expanding the items until no further dimensional adjustments are possible. In each iteration, the width W of the bin is reduced to

$$\max \left\{ z = \sum_{i \in \mathcal{I}} w_i \vartheta_i : z \leq W, \vartheta_i \in \{0, 1\}, i \in \mathcal{I} \right\}, \quad (5)$$

where w_i represents the width of item i and ϑ_i is a binary variable indicating whether item i is selected. A similar reduction is applied to the bin's height. To enlarge the items, the width w_i of each item $i \in \mathcal{I}$ is updated to $W - \overline{W}_i$, where

$$\overline{W}_i = \max \left\{ z = \sum_{j \in \mathcal{I} \setminus \{i\}} w_j \vartheta_j : z \leq W - w_i, \vartheta_j \in \{0, 1\}, j \in \mathcal{I} \setminus \{i\} \right\}. \quad (6)$$

A similar procedure is applied to adjust the heights of the items. These modifications to both bin and item dimensions can be efficiently computed using standard dynamic programming techniques.

Computing Initial Bounds. Next, we calculate three polynomial lower bounds from Boschetti and Mingozzi (2003a) and Carlier et al. (2007) (namely, $LB_{\text{BM},3}^2$, $LB_{\text{CCM},1}^2$, and $LB_{\text{CCM},2}^2$; see the classification of Serairi and Haouari (2018) and §EC.1.1 of the E-Companion for details) and take the maximum of them as the initial lower bound LB_0 on the minimal number of bins used. An initial upper bound value UB_0 is then computed using the hybrid heuristic introduced in §4.1. If $LB_0 = UB_0$, the initial solution is proven to be optimal, and the algorithm ends. Otherwise, we execute subsequent BPC enumeration to solve the MP.

Solving the Root Node. Let LB and UB denote the current best lower and upper bounds associated with the BPC enumeration tree, initialized to LB_0 and UB_0 , respectively. Each BPC node uses the column-and-row generation (CRG) described in the next section to solve the LP relaxation of the MP (LMP), obtained by relaxing constraints (2c) to $\xi_p \geq 0$ for every pattern $p \in \mathcal{P}$. We first address the root node, where the rounded-up values of the numerically-safe lower bounds on the optimal LMP solution value computed by CRG are used to update LB . Once $UB = LB$ or the optimal LMP solution at the root node is not fractional, the incumbent (primal) solution is proven to be optimal.

Branching Scheme. If the optimal LMP solution at the root node is fractional and $UB > LB$, we employ the well-known *Ryan-Foster* (RF) branching scheme (see Ryan and Foster (1981) for details) to generate two child nodes for the root node (also for all other nodes with a fractional optimal LMP solution) by requiring a selected pair of items to be packed either in the same bin or in different bins. In the following, these requirements are referred to as RF branching constraints for simplicity.

Search Strategy. To quickly obtain better incumbent solutions (i.e., reduce UB), we first apply a diving heuristic at the root node to partially explore the BPC tree (see §4.2), followed by a classic depth-first search to complete the exploration. If all generated nodes are solved within the time limit, the incumbent solution is proven to be optimal, and LB is improved to UB . Otherwise, an absolute optimality gap of $UB - LB$ is obtained. Since the initial bounds are computed using integers and LB is updated with numerically safe lower bounds in CRG, the proposed BPC algorithm can be considered numerically exact.

3. Column-and-Row Generation

For a generic BPC node, the CRG is executed iteratively based on a restricted LMP (RLMP), which is modified from LMP by replacing $\mathcal{P}$ with its subset $\overline{\mathcal{P}}$, $\mathcal{I}$ with its subset $\overline{\mathcal{I}}$, and $\widetilde{LB}$ in the VB cut with a local lower bound $\overline{LB}$. Specifically, at the root node, $\overline{\mathcal{P}}$, $\overline{\mathcal{I}}$, and $\overline{LB}$ are initialized, respectively, to a set of patterns consisting of the initial solution and each item in $\mathcal{I}$, the empty set, and LB_0 . At other nodes, $\overline{\mathcal{I}}$ and $\overline{LB}$ are initialized to the empty set and the local lower bound of their parent nodes, respectively; $\overline{\mathcal{P}}$ is initialized to the set of patterns from their parents that satisfy the RF branching constraints.

For brevity, we refer to the LMP and RLMP at the generic node as the *current* LMP and RLMP. In each iteration, a rounding heuristic (see §4.3) is first applied to try to find a better incumbent solution. We then address the so-called *Pricing Problem* (PP) (see §3.1) through a pricing algorithm (see §3.3) that updates the local lower bound $\overline{LB}$ by computing numerically safe bounds and searches for a set $\mathcal{P}'$ of patterns with negative reduced cost. If $\overline{LB}$ is improved by the pricing algorithm, the VB cut is accordingly changed by the updated $\overline{LB}$. Additionally, if $\mathcal{P}' \neq \emptyset$, the newly generated patterns and their corresponding columns are added to $\overline{\mathcal{P}}$ and the current RLMP, respectively. Otherwise, if $\mathcal{P}'$ is empty, we apply the rounding heuristic again and a primal heuristic that solves problem $\text{USC}(\overline{\mathcal{P}}_u^{10^4}, UB - 1)$ using a commercial solver (CPLEX) with a time limit of 0.5 seconds, where $\overline{\mathcal{P}}_u$ is a global pattern set consisting of the union of the patterns found in the hybrid heuristic (§4.1) and $\overline{\mathcal{P}}$ of all nodes that have been explored or are being explored, $\overline{\mathcal{P}}_u^{10^4} \subseteq \overline{\mathcal{P}}_u$ is the set of at most 10^4 patterns that were recently generated, and $\text{USC}(\tilde{\mathcal{P}}, U)$ is a set-covering like model defined on an arbitrary pattern set $\tilde{\mathcal{P}}$ and limited by U defined as follows:

$$(\text{USC}(\tilde{\mathcal{P}}, U)) \quad \min \quad \sum_{p \in \tilde{\mathcal{P}}} \xi_p, \quad (7a)$$

$$\text{s. t.} \quad \sum_{p \in \tilde{\mathcal{P}}} a_{ip} \xi_p \geq 1, \quad \forall i \in \mathcal{I}, \quad (7b)$$

$$\sum_{p \in \tilde{\mathcal{P}}} \xi_p \leq U, \quad (7c)$$

$$\xi_p \in \{0, 1\}, \quad \forall p \in \tilde{\mathcal{P}}. \quad (7d)$$

Solving $\text{USC}(\overline{\mathcal{P}}_u^{10^4}, UB - 1)$ aims to reduce UB by finding a better primal solution with a cost of at most $UB - 1$ by selecting a feasible subset of at most $UB - 1$ patterns from $\overline{\mathcal{P}}_u^{10^4}$. Finally, if $\overline{LB} = UB - 1$, row generation is performed: all violated SR cuts are identified by complete enumeration, and up to 1,000 most violated cuts are added to the RLMP. If no violated SR cut is found or $\overline{LB} < UB - 1$, the current CRG iteration ends; otherwise, the next CRG iteration begins.

3.1. Pricing Problem

Let $\mu = (\mu_1, \dots, \mu_n)$, $\lambda = (\lambda_1, \dots, \lambda_{|\overline{\mathcal{I}}|})$, and γ be the dual variables of the current RLMP. The vector $\mu \geq 0$ is associated with constraints (2b), the vector $\lambda \leq 0$ with constraints (3), and the scalar $\gamma \geq 0$ with

constraint (4). For a dual solution (μ, λ, γ) , a pattern $p \in \mathcal{P}$, and a parameter b (which is equal to 1 here; different values are used later), the *reduced cost* of pattern p (column ξ_p) is

$$\delta_p(b, \mu, \lambda, \gamma) = b - \gamma - \sum_{i \in \mathcal{I}_p} \mu_i - \sum_{t \in \overline{\mathcal{T}}(p)} \lambda_t, \quad (8)$$

where $\overline{\mathcal{T}}(p) = \{t \in \overline{\mathcal{T}} : |\mathcal{I}_p \cap \mathcal{T}_t| \geq 2\}$, i.e., the subset of triplets containing at least two items of pattern p .

The PP aims to identify at least one pattern $p \in \mathcal{P} \setminus \overline{\mathcal{P}}$ associated with a column ξ_p whose reduced cost $\delta_p(b, \mu, \lambda, \gamma)$ is negative. If such a pattern exists, the column ξ_p is added to the current RLMP. Otherwise, the solution (μ, λ, γ) is proven to be a feasible dual solution of the current LMP.

Furthermore, for each item $i \in \mathcal{I}$, we define $\mathcal{X}_i \subseteq \mathbb{N} \cap [0, W - w_i]$ and $\mathcal{Y}_i \subseteq \mathbb{N} \cap [0, H - h_i]$ as the subsets of potential horizontal and vertical coordinates (representing the bottom-left corner positions), respectively. The set of all possible horizontal positions for all items in $\mathcal{I}$ is denoted by $\mathcal{X} = \bigcup_{i \in \mathcal{I}} \mathcal{X}_i$, which aggregates the potential horizontal positions for every item. For each item $i \in \mathcal{I}$ and coordinate $x \in \mathcal{X}_i$, we define q_{ix} as a binary variable, equal to 1 if and only if the bottom-left corner of item i is placed at position x on the horizontal axis. Additionally, for each item $i \in \mathcal{I}$, we introduce $\hat{y}_i$, a non-negative integer variable representing its bottom-left corner position on the vertical axis. Given a vector $\mathbf{q} \in \{0, 1\}^{\sum_{i \in \mathcal{I}} |\mathcal{X}_i|}$ representing the x -coordinates of the items, we define $\mathcal{X}_i^x$ as the subset of the horizontal points for item i corresponding to the positions where the item also occupies position x . The set of items packed into the bin and occupying a position $x \in \mathcal{X}$ can be expressed as follows:

$$\mathcal{I}(x, \mathbf{q}) = \left\{ i \in \mathcal{I} : \sum_{\tilde{x} \in \mathcal{X}_i^x} q_{i\tilde{x}} = 1 \right\} \quad \text{where} \quad \mathcal{X}_i^x = \left\{ x_i \in \mathcal{X}_i : x - w_i + 1 \leq x_i \leq x \right\}. \quad (9)$$

For each triplet $t \in \overline{\mathcal{T}}$, let ψ_t be a binary variable that equals one if at least two items from the triplet are selected in the solution and zero otherwise. To represent the constraints imposed by RF branching, we define two undirected graphs: the *same-bin* graph $\mathcal{G}_s = (\mathcal{I}, \mathcal{E}_s)$ and the *different-bin* graph $\mathcal{G}_d = (\mathcal{I}, \mathcal{E}_d)$. An edge $\{i, j\} \in \mathcal{E}_s$ forces items i and j to be packed in the same bin, whereas an edge $\{i, j\} \in \mathcal{E}_d$ forbids them from being packed together, thereby defining a conflict. Following the formulation proposed by Côté et al. (2014) for the Two-Dimensional Strip-Packing Problem, the PP can be modeled as follows:

$$(\text{PP}(b, \mu, \lambda, \gamma)) \quad \min \quad b - \gamma - \sum_{i \in \mathcal{I}} \sum_{x \in \mathcal{X}_i} \mu_i q_{ix} - \sum_{t \in \overline{\mathcal{T}}} \lambda_t \psi_t, \quad (10a)$$

$$\text{s. t.} \quad \sum_{x \in \mathcal{X}_i} q_{ix} \leq 1, \quad \forall i \in \mathcal{I}, \quad (10b)$$

$$\sum_{x \in \mathcal{X}_i} q_{ix} - \sum_{x \in \mathcal{X}_j} q_{jx} = 0, \quad \forall \{i, j\} \in \mathcal{E}_s, \quad (10c)$$

$$\sum_{x \in \mathcal{X}_i} q_{ix} + \sum_{x \in \mathcal{X}_j} q_{jx} \leq 1, \quad \forall \{i, j\} \in \mathcal{E}_d, \quad (10d)$$

$$\sum_{x \in \mathcal{X}_i} q_{ix} + \sum_{x \in \mathcal{X}_j} q_{jx} - \psi_t \leq 1, \quad \forall t \in \overline{\mathcal{T}}, i, j \in \mathcal{T}_t, i < j, \quad (10e)$$

$$\bigcap_{i \in \mathcal{I}(x, \mathbf{q})} [\hat{y}_i, \hat{y}_i + h_i] = \emptyset, \quad \forall x \in \mathcal{X}, \quad (10f)$$

$$q_{ix} \in \{0, 1\}, \quad \forall i \in \mathcal{I}, x \in \mathcal{X}_i, \quad (10g)$$

$$\hat{y}_i \in \mathcal{Y}_i, \quad \forall i \in \mathcal{I}, \quad (10h)$$

$$\psi_t \in \{0, 1\}, \quad \forall t \in \overline{\mathcal{T}}. \quad (10i)$$

The objective function (10a) minimizes the reduced cost. Constraints (10b) ensure that each item is packed at most once. Constraints (10c)-(10d) enforce the RF branching constraints from the set of edges $\mathcal{E}_s$ and $\mathcal{E}_d$ of graphs $\mathcal{G}_s$ and $\mathcal{G}_d$, respectively. Constraints (10e) link the value of the binary variables of the triplets to those of the items. As a result, according to (8), the value $\sum_{t \in \overline{\mathcal{T}}} \lambda_t \psi_t$ must be subtracted from the reduced cost value of a pattern. To prevent overlaps, Constraints (10f) ensure that for each position $x \in \mathcal{X}$, the vertical intervals $[\hat{y}_i, \hat{y}_i + h_i]$, corresponding to items occupying the same position x , do not overlap. When solving 2BPP-G, the PP additionally requires that the patterns corresponding to $\mathbf{q}$ and $\hat{\mathbf{y}}$ are guillotine-cuttable. The following discusses how to solve the PP with and without guillotine constraints, respectively.

3.2. Solving the Pricing Problem

To effectively address the PP without guillotine constraints, Wang et al. (2025a) explored a Generalized Two-Dimensional Knapsack Problem (G2KP), which extends the classic Two-Dimensional Knapsack Problem by introducing RF branching constraints ($\mathcal{G}_s$ and $\mathcal{G}_d$) and penalties (λ' and $\overline{\mathcal{T}}$) as follows:

$$(\text{G2KP}(\boldsymbol{\mu}, \boldsymbol{\lambda})) \quad \max \quad \sum_{i \in \mathcal{I}} \sum_{x \in \mathcal{X}_i} \mu_i q_{ix} - \sum_{t \in \overline{\mathcal{T}}} \lambda'_t \psi_t, \quad (11a)$$

$$\text{s. t.} \quad (10b) - (10i), \quad (11b)$$

where $\lambda' = -\lambda$ and $\boldsymbol{\mu}$ represents penalties and item profits, respectively. The objective function (11a) maximizes the sum of the profits of selected items minus the penalties associated with selected triplets. Since $(b - \gamma)$ minus the optimal solution value of $\text{G2KP}(\boldsymbol{\mu}, \boldsymbol{\lambda})$ equals the corresponding optimal solution cost of $\text{PP}(b, \boldsymbol{\mu}, \boldsymbol{\lambda}, \gamma)$, hence, solving any PP is equivalent to solving the corresponding G2KP.

For solving G2KP, Wang et al. (2025a) designed a numerically exact algorithm (NEA), which does not rely on floating-point solvers and can be considered a three-level BB method. The first level selects items to be packed, while the second and third levels assign the x and y coordinates of the bottom-left corner of each selected item, respectively, ensuring the orthogonal packing is met. To accelerate the algorithm, the recent *Meet-In-the-Middle* technique proposed by Côté and Iori (2018) is used to compute smaller point sets $\mathcal{X}_i$ and $\mathcal{Y}_i$ than the ordinary point sets $[0, W - w_i] \cap \mathbb{N}$ and $[0, H - h_i] \cap \mathbb{N}$ for each item $i \in \mathcal{I}$.

We quote the techniques from Wang et al. (2025a) (see §3.1.2 and §3.3 of their paper) as follows: (i) Let $z_{\text{G2KP}}^*(\boldsymbol{\mu}, \boldsymbol{\lambda}, z_1, z_2)$ denote the optimal solution value of $\text{G2KP}(\boldsymbol{\mu}, \boldsymbol{\lambda})$ restricted to the range $[z_1, z_2]$, computed by the NEA; (ii) Let $\bar{z}_{\text{G2KP}}^{\text{dp}}(\boldsymbol{\mu})$ and $\bar{z}_{\text{G2KP}}^{\text{ls}}(\boldsymbol{\mu}, \boldsymbol{\lambda})$ denote two upper bounds on the optimal solution value of $\text{G2KP}(\boldsymbol{\mu}, \boldsymbol{\lambda})$, computed by the enhanced 0-1 knapsack function and the label-setting algorithm, respectively; (iii) Let $\bar{z}_{\text{G2KP}}(\boldsymbol{\mu}, \boldsymbol{\lambda})$ denote the upper bound on the optimal solution value of $\text{G2KP}(\boldsymbol{\mu}, \boldsymbol{\lambda})$, computed as $\bar{z}_{\text{G2KP}}^{\text{ls}}(\boldsymbol{\mu}, \boldsymbol{\lambda})$ or $\bar{z}_{\text{G2KP}}^{\text{dp}}(\boldsymbol{\mu})$ when the computation of $\bar{z}_{\text{G2KP}}^{\text{ls}}$ exceeds the memory limit; and (iv) Let

$\underline{z}_{\text{G2KP}}(\boldsymbol{\mu}, \boldsymbol{\lambda})$ denote a valid lower bound on the optimal value of $\text{G2KP}(\boldsymbol{\mu}, \boldsymbol{\lambda})$, computed by the label-setting algorithm. From this, we can deduce $\underline{z}_{\text{G2KP}}(\boldsymbol{\mu}, \boldsymbol{\lambda}) \leq z_{\text{G2KP}}^*(\boldsymbol{\mu}, \boldsymbol{\lambda}) \leq \bar{z}_{\text{G2KP}}(\boldsymbol{\mu}, \boldsymbol{\lambda})$, where $z_{\text{G2KP}}^*(\boldsymbol{\mu}, \boldsymbol{\lambda})$ denotes the optimal solution value of $\text{G2KP}(\boldsymbol{\mu}, \boldsymbol{\lambda})$. Notably, avoiding floating-point LP solvers lets all integer inputs be handled in fixed-point arithmetic, ensuring numerically safe bounds.

Although Wang et al. (2025a) did not consider the guillotine constraints, we adapt their approach by replacing the skyline-based heuristic with the BFH1, a single-bin guillotine packing heuristic introduced in Wang et al. (2025b), and by removing nodes that do not correspond to guillotine-cuttable patterns in the third-level enumeration tree of the NEA. For convenience, we use the above notation uniformly, although the meanings of the symbols differ slightly when solving instances of different problems.

3.3. Workflow of the Pricing Algorithm

This section outlines the workflow of the proposed pricing algorithm, which proceeds through four main steps. First, dual solutions are scaled to improve numerical stability. Second, numerically safe lower bounds are computed to enable early termination. Third, heuristic pricing is applied, both with and without dual cuts, to efficiently identify promising columns. Finally, exact pricing is performed within a reduced search range, ensuring correctness while maintaining computational efficiency.

Step 1: Scaling Dual Solutions. We first compute an optimal dual solution $(\boldsymbol{\mu}^*, \boldsymbol{\lambda}^*, \gamma^*)$ of the current RLMP using CPLEX and its corresponding *scaled integer* dual solution $(\boldsymbol{\mu}^{\text{int}}, \boldsymbol{\lambda}^{\text{int}}, \gamma^{\text{int}})$ by a scaling factor $K \in \mathbb{N}^+$, where $\mu_i^{\text{int}} = \lfloor K\mu_i^* \rfloor$ for all $i \in \mathcal{I}$, $\lambda_t^{\text{int}} = \lfloor K\lambda_t^* \rfloor$ for all $t \in \overline{\mathcal{T}}$, and $\gamma^{\text{int}} = \lfloor K\gamma^* \rfloor$. The *diminished* dual solution of $(\boldsymbol{\mu}^{\text{int}}, \boldsymbol{\lambda}^{\text{int}}, \gamma^{\text{int}})$ is calculated as $(\boldsymbol{\mu}^{\text{dim}}, \boldsymbol{\lambda}^{\text{dim}}, \gamma^{\text{dim}})$, where $\mu_i^{\text{dim}} = \frac{1}{K}\mu_i^{\text{int}}$ for all $i \in \mathcal{I}$, $\lambda_t^{\text{dim}} = \frac{1}{K}\lambda_t^{\text{int}}$ for all $t \in \overline{\mathcal{T}}$, and $\gamma^{\text{dim}} = \frac{1}{K}\gamma^{\text{int}}$. It can be easily seen that $(\boldsymbol{\mu}^{\text{dim}}, \boldsymbol{\lambda}^{\text{dim}}, \gamma^{\text{dim}})$ is a feasible dual solution of the RLMP of cost $z^{\text{dim}} = \sum_{i \in \mathcal{I}} \mu_i^{\text{dim}} + \sum_{t \in \overline{\mathcal{T}}} \lambda_t^{\text{dim}} + \overline{LB}\gamma^{\text{dim}}$. The following proposition holds.

Proposition 1 *Let $(\boldsymbol{\mu}^*, \boldsymbol{\lambda}^*, \gamma^*)$ be an error-free optimal dual solution to the RLMP of cost z_{RLMP}^* . Then,*

$$z_{\text{RLMP}}^* - \frac{1}{K}(|\mathcal{I}| + |\overline{\mathcal{T}}| + \overline{LB}) \leq z^{\text{dim}} \leq z_{\text{RLMP}}^*.$$

Proof. We have

$$z_{\text{RLMP}}^* - z^{\text{dim}} = \sum_{i \in \mathcal{I}} (\mu_i^* - \mu_i^{\text{dim}}) + \sum_{t \in \overline{\mathcal{T}}} (\lambda_t^* - \lambda_t^{\text{dim}}) + \overline{LB}(\gamma^* - \gamma^{\text{dim}}). \quad (12)$$

Furthermore, since $0 \leq r - \frac{1}{K} \lfloor Kr \rfloor \leq \frac{1}{K}$ hold for any $r \in \mathbb{R}$, we have

$$\mu_i^* - \mu_i^{\text{dim}} \in [0, \frac{1}{K}], \quad \lambda_t^* - \lambda_t^{\text{dim}} \in [0, \frac{1}{K}], \quad \text{and} \quad \gamma^* - \gamma^{\text{dim}} \in [0, \frac{1}{K}] \quad \text{hold for all } i \in \mathcal{I}, t \in \overline{\mathcal{T}}, \quad (13)$$

hence, $0 \leq z_{\text{RLMP}}^* - z^{\text{dim}} \leq \frac{1}{K}(|\mathcal{I}| + |\overline{\mathcal{T}}| + \overline{LB}) \implies z_{\text{RLMP}}^* - \frac{1}{K}(|\mathcal{I}| + |\overline{\mathcal{T}}| + \overline{LB}) \leq z^{\text{dim}} \leq z_{\text{RLMP}}^*$. $\square$

Therefore, the value of K should be as large as possible to avoid deriving unnecessarily weak safe dual bounds (see Propositions 3 and 4). However, using an infinite value of K is not practical because the scaled dual solution and the integers generated in the subsequent computations related to it are stored as 64-bit signed integers, which support minimum and maximum values of -2^{63} and $2^{63} - 1$, respectively. Due

to the definition of the reduced cost $\delta_p(K, \boldsymbol{\mu}^{\text{int}}, \boldsymbol{\lambda}^{\text{int}}, \gamma^{\text{int}})$ and since $\boldsymbol{\mu}^{\text{int}} \geq \mathbf{0}$, $\boldsymbol{\lambda}^{\text{int}} \leq \mathbf{0}$, and $\gamma^{\text{int}} \geq 0$, valid lower and upper bounds on the optimal solution cost of $\text{PP}(K, \boldsymbol{\mu}^{\text{int}}, \boldsymbol{\lambda}^{\text{int}}, \gamma^{\text{int}})$ are $K - \gamma^{\text{int}} - \sum_{i \in \mathcal{I}} \mu_i^{\text{int}}$ and $K - \gamma^{\text{int}} - \sum_{t \in \overline{\mathcal{T}}} \lambda_t^{\text{int}}$, respectively. Hence, to avoid numerical overflow, it is necessary to ensure that

$$K - \gamma^{\text{int}} - \sum_{i \in \mathcal{I}} \mu_i^{\text{int}} \geq -2^{63} \quad \text{and} \quad K - \gamma^{\text{int}} - \sum_{t \in \overline{\mathcal{T}}} \lambda_t^{\text{int}} \leq 2^{63} - 1. \quad (14)$$

As analyzed in §EC.2.2 of the E-Companion, setting K to $10^{11} \approx 2^{37}$ yields good results and prevents numerical overflow, ensuring that the conditions in (14) are always satisfied in our experiments. We observe that numerical errors can result in very small negative values (e.g., -10^{-12}) for variables that should be non-negative, such as $\boldsymbol{\mu}$ and γ , and very small positive values (e.g., 10^{-12}) for variables that should be non-positive, such as $\boldsymbol{\lambda}$. After scaling, these insignificant values may become large enough to violate their domain constraints; for example, causing $\boldsymbol{\mu}^{\text{int}} < \mathbf{0}$ or $\boldsymbol{\lambda}^{\text{int}} > \mathbf{0}$. To address this issue, we examine the scaled dual solution and set any values that violate their domain constraints to zero.

Step 2: Computing Numerically Safe Lower Bounds and Early Termination. To enable early termination of the CRG algorithm and thereby reduce unnecessary computational effort, we employ two fathoming rules that rely on computing a valid and numerically-safe lower bound on the optimal solution cost of the $\text{PP}(K, \boldsymbol{\mu}^{\text{int}}, \boldsymbol{\lambda}^{\text{int}}, \gamma^{\text{int}})$, as stated by the following proposition.

Proposition 2 *In an arbitrary CRG iteration, let $\tilde{z}_{\text{G2KP}}(\boldsymbol{\mu}^{\text{int}}, \boldsymbol{\lambda}^{\text{int}})$ be a valid and safe upper bound on the optimal solution cost of the $\text{G2KP}(\boldsymbol{\mu}^{\text{int}}, \boldsymbol{\lambda}^{\text{int}})$, computed by $\tilde{z}_{\text{G2KP}}$ or z_{G2KP}^* . Then, $K - \gamma^{\text{int}} - \tilde{z}_{\text{G2KP}}(\boldsymbol{\mu}^{\text{int}}, \boldsymbol{\lambda}^{\text{int}})$ is a valid and safe lower bound on the optimal solution cost of the $\text{PP}(K, \boldsymbol{\mu}^{\text{int}}, \boldsymbol{\lambda}^{\text{int}}, \gamma^{\text{int}})$.*

Proof. *In an arbitrary CRG iteration, recall that $\tilde{z}_{\text{G2KP}}(\boldsymbol{\mu}^{\text{int}}, \boldsymbol{\lambda}^{\text{int}})$ is defined as a valid and safe upper bound on the optimal solution cost of $\text{G2KP}(\boldsymbol{\mu}^{\text{int}}, \boldsymbol{\lambda}^{\text{int}})$. Hence, we can deduce that*

$$\tilde{z}_{\text{G2KP}}(\boldsymbol{\mu}^{\text{int}}, \boldsymbol{\lambda}^{\text{int}}) \geq \sum_{i \in \mathcal{I}} \sum_{x \in \mathcal{X}_i} \mu_i^{\text{int}} \varrho_{ix} + \sum_{t \in \overline{\mathcal{T}}} \lambda_t^{\text{int}} \psi_t, \quad (15a)$$

$$\implies K - \gamma^{\text{int}} - \tilde{z}_{\text{G2KP}}(\boldsymbol{\mu}^{\text{int}}, \boldsymbol{\lambda}^{\text{int}}) \leq K - \gamma^{\text{int}} - \sum_{i \in \mathcal{I}} \sum_{x \in \mathcal{X}_i} \mu_i^{\text{int}} \varrho_{ix} - \sum_{t \in \overline{\mathcal{T}}} \lambda_t^{\text{int}} \psi_t, \quad (15b)$$

which holds for all solutions $(\boldsymbol{\varrho}, \boldsymbol{\psi})$ feasible to $\text{G2KP}(\boldsymbol{\mu}^{\text{int}}, \boldsymbol{\lambda}^{\text{int}})$. Therefore, $K - \gamma^{\text{int}} - \tilde{z}_{\text{G2KP}}(\boldsymbol{\mu}^{\text{int}}, \boldsymbol{\lambda}^{\text{int}})$ is a valid lower bound on the optimal solution value of $\text{PP}(K, \boldsymbol{\mu}^{\text{int}}, \boldsymbol{\lambda}^{\text{int}}, \gamma^{\text{int}})$. Moreover, the lower bound is numerically safe because it relies on error-free fixed-point computations rather than floating-point computations. $\square$

Further, the following two propositions guarantee that valid and safe lower bounds on the optimal solution cost of the LMP at a generic iteration of the CRG algorithm can be computed.

Proposition 3 *In an arbitrary CRG iteration, if $\delta_p(K, \boldsymbol{\mu}^{\text{int}}, \boldsymbol{\lambda}^{\text{int}}, \gamma^{\text{int}}) \geq 0$ holds for all $p \in \mathcal{P} \setminus \overline{\mathcal{P}}$, then z^{dim} is a valid and numerically safe lower bound on the optimal solution value of the current LMP.*

Proof. In an arbitrary CRG iteration, if the condition $\delta_p(K, \boldsymbol{\mu}^{\text{int}}, \boldsymbol{\lambda}^{\text{int}}, \gamma^{\text{int}}) \geq 0$ holds for all $p \in \mathcal{P} \setminus \overline{\mathcal{P}}$, we can deduce that

$$\frac{1}{K} \delta_p(K, \boldsymbol{\mu}^{\text{int}}, \boldsymbol{\lambda}^{\text{int}}, \gamma^{\text{int}}) = \delta_p(1, \boldsymbol{\mu}^{\text{dim}}, \boldsymbol{\lambda}^{\text{dim}}, \gamma^{\text{dim}}) = 1 - \gamma^{\text{dim}} - \sum_{i \in \mathcal{I}_p} \mu_i^{\text{dim}} - \sum_{t \in \mathcal{T}(p)} \lambda_t^{\text{dim}} \geq 0 \quad (16)$$

and $\gamma^{\text{dim}} + \sum_{i \in \mathcal{I}_p} \mu_i^{\text{dim}} + \sum_{t \in \mathcal{T}(p)} \lambda_t^{\text{dim}} \leq 1$ holds for all $p \in \mathcal{P} \setminus \overline{\mathcal{P}}$. This implies that $(\boldsymbol{\mu}^{\text{dim}}, \boldsymbol{\lambda}^{\text{dim}}, \gamma^{\text{dim}})$ is a feasible solution with objective value $z^{\text{dim}} = \sum_{i \in \mathcal{I}} \mu_i^{\text{dim}} + \sum_{t \in \mathcal{T}} \lambda_t^{\text{dim}} + \overline{LB} \gamma^{\text{dim}}$ to the dual of the current LMP. Therefore, by the weak duality theorem, z^{dim} is a valid lower bound on the optimal value of the current LMP. In addition, since the determination of $\delta_p(K, \boldsymbol{\mu}^{\text{int}}, \boldsymbol{\lambda}^{\text{int}}, \gamma^{\text{int}}) \geq 0$ relies on integer values rather than floating points, thus avoiding cancellation errors, the lower bound z^{dim} is numerically safe. $\square$

Proposition 4 In an arbitrary CRG iteration, let $\tilde{\underline{\delta}}^{\text{int}} = K - \gamma^{\text{int}} - \tilde{z}_{\text{G2KP}}(\boldsymbol{\mu}^{\text{int}}, \boldsymbol{\lambda}^{\text{int}}) < 0$ be a valid and safe lower bound on the optimal solution cost of $\text{PP}(K, \boldsymbol{\mu}^{\text{int}}, \boldsymbol{\lambda}^{\text{int}}, \gamma^{\text{int}})$, proven as in Proposition 2. Then, $\frac{z^{\text{dim}}}{1 - (\tilde{\underline{\delta}}^{\text{int}}/K)}$ is a valid and safe lower bound on the optimal solution value of the current LMP.

Proof. In an arbitrary CRG iteration, based on (15b), we can deduce that

$$\frac{K - \gamma^{\text{int}} - \tilde{z}_{\text{G2KP}}(\boldsymbol{\mu}^{\text{int}}, \boldsymbol{\lambda}^{\text{int}})}{K} \leq \frac{K - \gamma^{\text{int}} - \sum_{i \in \mathcal{I}} \sum_{x \in \mathcal{X}_i} \mu_i^{\text{int}} \varrho_{ix} - \sum_{t \in \mathcal{T}} \lambda_t^{\text{int}} \psi_t}{K}, \quad (17a)$$

$$\Rightarrow \frac{K - \gamma^{\text{int}} - \tilde{z}_{\text{G2KP}}(\boldsymbol{\mu}^{\text{int}}, \boldsymbol{\lambda}^{\text{int}})}{K} \leq 1 - \gamma^{\text{dim}} - \sum_{i \in \mathcal{I}} \sum_{x \in \mathcal{X}_i} \mu_i^{\text{dim}} \varrho_{ix} - \sum_{t \in \mathcal{T}} \lambda_t^{\text{dim}} \psi_t, \quad (17b)$$

$$\Rightarrow \gamma^{\text{dim}} + \sum_{i \in \mathcal{I}} \sum_{x \in \mathcal{X}_i} \mu_i^{\text{dim}} \varrho_{ix} + \sum_{t \in \mathcal{T}} \lambda_t^{\text{dim}} \psi_t \leq 1 - \frac{K - \gamma^{\text{int}} - \tilde{z}_{\text{G2KP}}(\boldsymbol{\mu}^{\text{int}}, \boldsymbol{\lambda}^{\text{int}})}{K}. \quad (17c)$$

Recall that $\tilde{\underline{\delta}}^{\text{int}}$ is defined as $\tilde{\underline{\delta}}^{\text{int}} = K - \gamma^{\text{int}} - \tilde{z}_{\text{G2KP}}(\boldsymbol{\mu}^{\text{int}}, \boldsymbol{\lambda}^{\text{int}}) < 0$. Hence, $1 - \frac{\tilde{\underline{\delta}}^{\text{int}}}{K} > 0$. According to (17c), we have

$$\frac{\gamma^{\text{dim}} + \sum_{i \in \mathcal{I}} \sum_{x \in \mathcal{X}_i} \mu_i^{\text{dim}} \varrho_{ix} + \sum_{t \in \mathcal{T}} \lambda_t^{\text{dim}} \psi_t}{1 - \frac{\tilde{\underline{\delta}}^{\text{int}}}{K}} \leq \frac{1 - \frac{\tilde{\underline{\delta}}^{\text{int}}}{K}}{1 - \frac{\tilde{\underline{\delta}}^{\text{int}}}{K}}, \quad (18a)$$

$$\Rightarrow 1 - \frac{\gamma^{\text{dim}}}{1 - \frac{\tilde{\underline{\delta}}^{\text{int}}}{K}} - \sum_{i \in \mathcal{I}} \sum_{x \in \mathcal{X}_i} \frac{\mu_i^{\text{dim}}}{1 - \frac{\tilde{\underline{\delta}}^{\text{int}}}{K}} \varrho_{ix} - \sum_{t \in \mathcal{T}} \frac{\lambda_t^{\text{dim}}}{1 - \frac{\tilde{\underline{\delta}}^{\text{int}}}{K}} \psi_t \geq 0, \quad (18b)$$

which holds for all feasible solutions $(\boldsymbol{\varrho}, \boldsymbol{\psi})$. This means that $\delta_p\left(1, \frac{\boldsymbol{\mu}^{\text{dim}}}{1 - \frac{\tilde{\underline{\delta}}^{\text{int}}}{K}}, \frac{\boldsymbol{\lambda}^{\text{dim}}}{1 - \frac{\tilde{\underline{\delta}}^{\text{int}}}{K}}, \frac{\gamma^{\text{dim}}}{1 - \frac{\tilde{\underline{\delta}}^{\text{int}}}{K}}\right) \geq 0$ holds for all $p \in \mathcal{P} \setminus \overline{\mathcal{P}}$. Therefore, $\left(\frac{\boldsymbol{\mu}^{\text{dim}}}{1 - \frac{\tilde{\underline{\delta}}^{\text{int}}}{K}}, \frac{\boldsymbol{\lambda}^{\text{dim}}}{1 - \frac{\tilde{\underline{\delta}}^{\text{int}}}{K}}, \frac{\gamma^{\text{dim}}}{1 - \frac{\tilde{\underline{\delta}}^{\text{int}}}{K}}\right)$ is a feasible solution to the dual of the current LMP. Based on the weak duality theorem, the value of the feasible dual solution, $\frac{1}{1 - \frac{\tilde{\underline{\delta}}^{\text{int}}}{K}} (\sum_{i \in \mathcal{I}} \mu_i^{\text{dim}} + \sum_{t \in \mathcal{T}} \lambda_t^{\text{dim}} + \overline{LB} \gamma^{\text{dim}}) = \frac{z^{\text{dim}}}{1 - \frac{\tilde{\underline{\delta}}^{\text{int}}}{K}}$, is a valid lower bound on the optimal solution value of the current LMP. Since the computation of the lower bound relies on integer values rather than floating points, thus avoiding cancellation errors, $\frac{z^{\text{dim}}}{1 - \frac{\tilde{\underline{\delta}}^{\text{int}}}{K}}$ is numerically safe. $\square$

Specifically, we compute $\underline{\delta}^{\text{int}} = K - \gamma^{\text{int}} - \tilde{z}_{\text{G2KP}}(\boldsymbol{\mu}^{\text{int}}, \boldsymbol{\lambda}^{\text{int}})$, and calculate the rounded value of the numerically safe bound to improve the current local lower bound $\overline{LB}$ based on Propositions 3 and 4: If $\underline{\delta}^{\text{int}} \geq 0$

(i.e., $\delta_p(K, \boldsymbol{\mu}^{\text{int}}, \boldsymbol{\lambda}^{\text{int}}, \gamma^{\text{int}}) \geq 0$ holds for all $p \in \mathcal{P} \setminus \overline{\mathcal{P}}$), $\overline{LB} = \max \left\{ \overline{LB}, \lceil z^{\text{dim}} \rceil \right\}$; otherwise, if $\underline{\delta}^{\text{int}} < 0$, $\overline{LB} = \max \left\{ \overline{LB}, \left\lceil \frac{z^{\text{dim}}}{1 - (\underline{\delta}^{\text{int}}/K)} \right\rceil \right\}$. Then, the following two fathoming rules are applied.

Fathoming 1 If $\overline{LB} \geq UB$, then the subproblem associated with the current node of the enumeration tree is fathomed by the upper bound UB .

Fathoming 2 If $\overline{LB} \geq \lceil z^{\text{dim}} \rceil$ or $\underline{\delta}^{\text{int}} \geq 0$, then the CRG procedure is halted, and a valid lower bound $\overline{LB}$ at the node is determined.

Step 3: Heuristic Pricing with and without Dual Cuts. Before executing the exact pricing, heuristic pricing is employed to quickly generate multiple patterns with negative reduced costs, thereby accelerating the convergence of column generation. Another promising direction to expedite convergence is to stabilize the column generation process through *dual cuts* (see, e.g. Ben Amor et al. 2006, Gschwind and Irnich 2016, da Silva and Schouery 2023, Guo et al. 2025). In our work, the following dual problem of the current RLMP with additional cuts is solved to derive an alternative dual solution $(\tilde{\boldsymbol{\mu}}, \tilde{\boldsymbol{\lambda}}, \tilde{\gamma})$:

$$\max \quad \sum_{i \in \mathcal{I}} \tilde{\mu}_i + \sum_{t \in \mathcal{T}} \tilde{\lambda}_t + \overline{LB} \tilde{\gamma}, \quad (19a)$$

$$\text{s. t.} \quad \sum_{i \in \mathcal{I}_p} \tilde{\mu}_i + \sum_{t \in \mathcal{T}: |\mathcal{I}_p \cap \mathcal{T}_t| \geq 2} \tilde{\lambda}_t + \tilde{\gamma} \leq 1, \quad \forall p \in \overline{\mathcal{P}}, \quad (19b)$$

$$\tilde{\mu}_{i-1} \geq \tilde{\mu}_i, \quad \forall i \in \mathcal{I} \setminus \{1\}, \quad (19c)$$

$$\tilde{\mu}_i \geq 0, \quad \forall i \in \mathcal{I}, \quad (19d)$$

$$\tilde{\lambda}_t \leq 0, \quad \forall t \in \mathcal{T}, \quad (19e)$$

$$\tilde{\gamma} \geq 0, \quad (19f)$$

where constraints (19c) represent a group of dual cuts. Recall that $\mathcal{I}$ is initially sorted in non-increasing order of item area. Hence, these cuts enforce the area of each item to be proportional to its associated dual variable $\tilde{\mu}$. According to Theorem 2 in Gschwind and Irnich (2016), if a dual cut $\tilde{\mu}_i \geq \tilde{\mu}_j$ is dual-optimal for BPP (the same holds for 2BPP(-G)), then the sum of the sizes of item $i \in \mathcal{I}$ and item $j \in \mathcal{I} \setminus \{i\}$ exceeds the size of the bin. This condition is not enforced for our dual cuts; thus, they are not necessarily dual-optimal. Nevertheless, as noted by da Silva and Schouery (2023), the high instability of dual solutions in column generation results in slow convergence, and focusing on stabilizing these values is intuitively efficient.

Specifically, let $(\tilde{\boldsymbol{\mu}}^{\text{int}}, \tilde{\boldsymbol{\lambda}}^{\text{int}}, \tilde{\gamma}^{\text{int}})$ be the integer dual solution scaled from an optimal solution to the problem (19) using the scaling factor K . We compute a valid and numerically safe lower bound $\tilde{\underline{\delta}}^{\text{int}} = K - \tilde{\gamma}^{\text{int}} - \overline{z}_{\text{G2KP}}(\tilde{\boldsymbol{\mu}}^{\text{int}}, \tilde{\boldsymbol{\lambda}}^{\text{int}})$ on the optimal solution cost of $\text{PP}(K, \tilde{\boldsymbol{\mu}}^{\text{int}}, \tilde{\boldsymbol{\lambda}}^{\text{int}}, \tilde{\gamma}^{\text{int}})$.

Let $\tilde{z}^{\dim} = \sum_{i \in \mathcal{I}} \tilde{\mu}_i^{\dim} + \sum_{t \in \mathcal{T}} \tilde{\lambda}_t^{\dim} + \overline{LB} \tilde{\gamma}^{\dim}$, where $(\tilde{\mu}^{\dim}, \tilde{\lambda}^{\dim}, \tilde{\gamma}^{\dim})$ is the corresponding diminished solution. If $\lceil \frac{\tilde{z}^{\dim}}{1 - (\underline{\delta}^{\text{int}}/K)} \rceil < UB$ and $\lceil \frac{\tilde{z}^{\dim}}{1 - (\underline{\delta}^{\text{int}}/K)} \rceil < \lceil \tilde{z}^{\dim} \rceil$, we compute $\underline{z}_{\text{G2KP}}(\tilde{\mu}^{\text{int}}, \tilde{\lambda}^{\text{int}})$ using the label-setting-based heuristic from Wang et al. (2025a). The execution condition refers to the aforementioned fathoming rules, aiming to reduce the number of computations of $\underline{z}_{\text{G2KP}}(\tilde{\mu}^{\text{int}}, \tilde{\lambda}^{\text{int}})$. Let $\mathcal{P}'$ be the set of patterns (having negative reduced costs relative to the solution $(\tilde{\mu}^{\text{int}}, \tilde{\lambda}^{\text{int}}, \tilde{\gamma}^{\text{int}})$) generated by the label-setting-based heuristic.

To accelerate the convergence of CRG without significantly increasing the complexity of RLMP, we limit the set to store at most $\max\{10, \lceil \frac{-100}{K} \underline{z}_{\text{G2KP}}(\tilde{\mu}^{\text{int}}, \tilde{\lambda}^{\text{int}}) \rceil\}$ patterns with the smallest reduced cost. If no patterns are stored, we compute $\underline{z}_{\text{G2KP}}(\mu^{\text{int}}, \lambda^{\text{int}})$ by means of a label-setting algorithm to attempt to fill $\mathcal{P}'$ with patterns having negative reduced cost. If $\mathcal{P}'$ is still empty, we resort to the patterns stored in the global pattern set $\overline{\mathcal{P}}_u$, traverse each pattern p in $\overline{\mathcal{P}}_u \setminus \overline{\mathcal{P}}$, and add p into $\mathcal{P}'$ if the pattern is feasible to the PP and $\delta_p(K, \mu^{\text{int}}, \lambda^{\text{int}}, \gamma^{\text{int}}) < 0$. If no patterns can be added, we resort to exact pricing in the next step; otherwise, we store at most $\max\{10, \lceil \frac{-100}{K} \min_{p \in \mathcal{P}'} \delta_p(K, \mu^{\text{int}}, \lambda^{\text{int}}, \gamma^{\text{int}}) \rceil\}$ patterns with the smallest reduced cost in $\mathcal{P}'$ and conclude the pricing algorithm by returning $\mathcal{P}'$.

Step 4: Exact Pricing with a Reduced Search Range. Exact pricing aims to determine whether a pattern in $\mathcal{P} \setminus \overline{\mathcal{P}}$ with a negative reduced cost exists. In our work, this objective can be achieved by solving $\text{G2KP}(\mu^{\text{int}}, \lambda^{\text{int}})$ using the NEA from Wang et al. (2025a), generating patterns with a reduced cost within $[\underline{\delta}^{\text{int}}, -1]$ by limiting the search range of the G2KP solution value to $[K - \gamma^{\text{int}} + 1, K - \gamma^{\text{int}} - \underline{\delta}^{\text{int}}]$. Through constraint (14), the lower bound of the search range will not cause numerical overflow (see (20a)), while the upper bound may exceed $2^{63} - 1$ (see (20b)). However, this has not been observed in our experiments.

$$K - \gamma^{\text{int}} - \sum_{i \in \mathcal{I}} \mu_i^{\text{int}} \geq -2^{63} \implies K - \gamma^{\text{int}} + 1 \geq -2^{63} + \sum_{i \in \mathcal{I}} \mu_i^{\text{int}} + 1 \geq -2^{63}, \quad (20a)$$

$$\begin{aligned} K - \gamma^{\text{int}} - \sum_{t \in \mathcal{T}} \lambda_t^{\text{int}} \leq 2^{63} - 1 &\implies K - \gamma^{\text{int}} - \underline{\delta}^{\text{int}} \leq 2^{63} - 1 + \sum_{t \in \mathcal{T}} \lambda_t^{\text{int}} - \underline{\delta}^{\text{int}} \\ &\leq 2^{63} - 1 + \sum_{t \in \mathcal{T}} \lambda_t^{\text{int}} - K + \gamma^{\text{int}} + \sum_{i \in \mathcal{I}} \mu_i^{\text{int}} \leq \sum_{i \in \mathcal{I}} \mu_i^{\text{int}}. \end{aligned} \quad (20b)$$

Note that, according to the equivalence relationship between PP and G2KP, any optimal reduced-cost pattern must be searched with a G2KP solution value within the search range $[K - \gamma^{\text{int}} + 1, K - \gamma^{\text{int}} - \underline{\delta}^{\text{int}}]$.

If there is no feasible solution of $\text{G2KP}(\mu^{\text{int}}, \lambda^{\text{int}})$ with value within the search range, we prove that $\delta_p(K, \mu^{\text{int}}, \lambda^{\text{int}}, \gamma^{\text{int}}) \geq 0$ holds for all $p \in \mathcal{P} \setminus \overline{\mathcal{P}}$ and thus $\overline{LB}$ can be updated to $\lceil z^{\dim} \rceil$ based on Proposition 3. Otherwise, the found patterns are added to $\mathcal{P}'$, and based on Proposition 4, $\overline{LB}$ can be updated to

$$\max \left\{ \overline{LB}, \left\lceil \frac{z^{\dim}}{1 - (z_{\text{G2KP}}^*(\mu^{\text{int}}, \lambda^{\text{int}}, K - \gamma^{\text{int}} + 1, K - \gamma^{\text{int}} - \underline{\delta}^{\text{int}})/K)} \right\rceil \right\}. \quad (21)$$

Subsequently, the fathoming rules 1 and 2 are applied. Recall that $z_{\text{G2KP}}^*(\mu, \lambda, z_1, z_2)$ denotes the optimal solution value of $\text{G2KP}(\mu, \lambda)$ restricted to the range $[z_1, z_2]$, computed by the NEA of Wang et al. (2025a).

Note again that, as explained above, this restriction does not exclude any optimal reduced-cost pattern.

However, in contrast to the above conventional exact pricing, we found that the search range $[K - \gamma^{\text{int}} + 1, K - \gamma^{\text{int}} - \underline{\delta}^{\text{int}}]$ can be further reduced to accelerate exact pricing. Specifically, let δ' be the smallest integer within the range $[\underline{\delta}^{\text{int}}, -1]$ such that $\lceil \frac{z^{\text{dim}}}{1 - (\delta'/K)} \rceil$ is less than both UB and $\lceil z^{\text{dim}} \rceil$, which can be efficiently computed using a standard binary search because $\lceil \frac{z^{\text{dim}}}{1 - (\delta'/K)} \rceil$ is a monotonic increasing function of δ' within the range $[\underline{\delta}^{\text{int}}, -1]$. Then, we solve the $\text{G2KP}(\mu^{\text{int}}, \lambda^{\text{int}})$ with the search range $[K - \gamma^{\text{int}} - \delta', K - \gamma^{\text{int}} - \underline{\delta}^{\text{int}}]$, narrowing the previous range by lifting the lower bound ($K - \gamma^{\text{int}} + 1$).

$$\delta' \in [\underline{\delta}^{\text{int}}, -1] \implies (K - \gamma^{\text{int}}) + \delta' \leq (K - \gamma^{\text{int}}) - 1 \implies K - \gamma^{\text{int}} + 1 \leq K - \gamma^{\text{int}} - \delta'. \quad (22)$$

Similarly, if there is no feasible solution of $\text{G2KP}(\mu^{\text{int}}, \lambda^{\text{int}})$ with value within the reduced search range, we prove that no pattern with a reduced cost exists within $[\underline{\delta}^{\text{int}}, \delta']$, i.e., $\delta' + 1$ is a valid lower bound on the optimal solution cost of $\text{PP}(K, \mu^{\text{int}}, \lambda^{\text{int}}, \gamma^{\text{int}})$. Therefore, $\overline{LB}$ is updated to $\lceil \frac{z^{\text{dim}}}{1 - (\delta' + 1)/K} \rceil$, which must be greater than or equal to UB or $\lceil z^{\text{dim}} \rceil$, thus either discarding the current node or terminating the current CRG iteration early. Otherwise, as before, the found patterns are added to $\mathcal{P}'$, and $\overline{LB}$ is updated to

$$\max \left\{ \overline{LB}, \left\lceil \frac{z^{\text{dim}}}{1 - (z_{\text{G2KP}}^*(\mu^{\text{int}}, \lambda^{\text{int}}, K - \gamma^{\text{int}} - \delta', K - \gamma^{\text{int}} - \underline{\delta}^{\text{int}})/K)} \right\rceil \right\}, \quad (23)$$

and the pricing algorithm ends by returning $\mathcal{P}'$. This enhanced strategy improves pruning by strengthening the lower bound and avoids searching for “useless” patterns with reduced costs in the range of $(\delta', -1]$, as identifying these patterns can be time-consuming.

3.4. Model Reduction via Waste Optimization

In the context of BPP, Delorme and Iori (2020), de Lima et al. (2023), and da Silva and Schouery (2023) observed that patterns with a negative reduced cost but large waste cannot improve the RLMP. Excluding these patterns from the RLMP can accelerate the algorithm without sacrificing optimality.

Following these works, we prohibit the generation of patterns with a wasted area greater than $WH \times z_{\text{RLMP}}^* - \sum_{i \in \mathcal{I}} w_i h_i$ at the root node, and prohibit the generation of patterns with a wasted area greater than $WH \times (UB - 1) - \sum_{i \in \mathcal{I}} w_i h_i$ at other nodes, where z_{RLMP}^* is the optimal solution value of the current RLMP. The above prohibition conflicts with the first reduction technique presented in Wang et al. (2025a), which we also use. Using them simultaneously would result in a loss of optimality; thus, it is not applicable in exact pricing. Further details and proofs can be found in the publications of these authors.

4. Primal Heuristics

To further enhance the proposed BPC algorithm, we also introduce three efficient primal heuristics: hybrid (§4.1), diving (§4.2), and rounding (§4.3) heuristics.

4.1. Hybrid Heuristic

The hybrid heuristic sequentially employs a fit-based heuristic, a triple-block-based heuristic, and a goal-driven heuristic to rapidly compute a high-quality initial 2BPP(-G) solution with a cost of UB_0 . The results in §5.1 demonstrate that the hybrid heuristic is highly competitive with leading 2BPP(-G) heuristics.

Fit-Based Heuristic. The fit-based heuristic operates in a first-fit outer manner, with a best-fit inner approach. In each iteration, a new bin is opened, and a best-fit packing heuristic is applied to pack as many unpacked items as possible into the new bin. Specifically, the skyline-based heuristic of Wang et al. (2025a) is used when solving 2BPP, and the BFH1 of Wang et al. (2025b) is used when solving 2BPP-G. The order of items affects the results of the fit-based heuristic; thus, it is applied in seven orders: original order, non-decreasing area (ties broken by non-decreasing width or height), non-decreasing perimeter (ties broken by non-decreasing width or height), non-decreasing width (ties broken by non-decreasing height), and non-decreasing height (ties broken by non-decreasing width).

Triple-Block-Based Heuristic. The triple-block-based heuristic is adapted from the 2BPP-G heuristic of Cui et al. (2018), including a greedy dynamic programming recursion for generating *triple-block patterns* and an integer programming approach based on model (7) to obtain an improved solution. Additionally, we introduce a post-improvement procedure to enhance the heuristic and incorporate the skyline-based heuristic from Wang et al. (2025a) to adapt 2BPP. Further details are provided in §EC.1.2 of the E-Companion.

Goal-Driven Iterated Local Search Heuristic. The goal-driven methods have been proven to effectively address 2BPP(-G), where solutions with the same number of bins are considered equivalent in quality, as demonstrated by Wei et al. (2013) and Gardeyn and Wauters (2022), which represent the state-of-the-art heuristics for 2BPP and 2BPP-G, respectively. Recently, Wang et al. (2025c) proposed a goal-driven iterated local search heuristic for a variant of 2BPP where the bins are identical circles, achieving notable results. Our goal-driven iterated local search heuristic for solving 2BPP(-G) is adapted from Wang et al. (2025c) and enhanced by a squeeze procedure. In brief, the heuristic iteratively searches for a feasible 2BPP(-G) solution that uses fewer bins than UB_0 . If such a solution is found, UB_0 is updated to the number of bins used by the new solution; otherwise, the squeeze procedure is applied to the incumbent solution in an attempt to obtain a more compact solution that uses fewer bins. Further details can be found in §EC.1.3 of the E-Companion.

4.2. Diving Heuristic

The diving heuristic, adapted from Wei et al. (2020b), aims to generate better primal solutions within the BP framework by exploring the search tree incompletely (Sadykov et al. 2019). Specifically, starting from the root node, the diving heuristic explores the BPC tree in a depth-first manner, limiting the search by removing nodes generated by at least two right branches. Unlike Wei et al. (2020b), during the diving process, we disable the computationally expensive exact pricing to explore more nodes.

4.3. Rounding Heuristic

The rounding heuristic is applied in an arbitrary CRG iteration. It takes the fractional solution of the current RLMP and its associated patterns as input, selecting a subset to construct a feasible primal solution for 2BPP(-G). If the constructed solution is better than the incumbent solution, the incumbent is updated

accordingly. The heuristic combines a deep adaptive strategy with a tabu search packing subroutine to allocate more computational effort to deeper nodes. Furthermore, we observe that the patterns generated by the rounding heuristic typically exhibit high utilization rates. To accelerate the convergence of the CRG algorithm, we add the patterns with negative reduced costs that meet the waste conditions described in §3.4 into the RLMP. A detailed description of the rounding heuristic is provided in §EC.1.4 of the E-Companion.

5. Computational Results

This section presents the main computational results of the proposed numerically exact BPC algorithm (referred to as “NE-BPC”) on 500 classic benchmark instances (see §5.1) and 500 newly generated large-scale instances (see §5.2). Additionally, Section 5.3 examines the effectiveness of the key techniques incorporated into NE-BPC. Section 5.4 analyzes the importance of the numerically safe lower bounds by testing a non-numerically safe version of NE-BPC. For further investigation, Section 5.5 uses an 8-hour time limit to attempt to close several open instances that failed to be solved within one hour. Further experiments and analyses are provided in Section EC.2 of the E-Companion. All algorithmic code, instance data, and detailed experimental results and solutions will be made available on GitHub following the peer review process.

The NE-BPC was implemented in JAVA and executed on a Windows 10 Professional (64-bit) system. The processor used was an Intel Core i5-10400F @ 2.9 GHz with 16 GB of RAM and a single-thread PassMark score of 2541 (<https://www.cpubenchmark.net>, 2025), which is used to compare the performance of different CPUs, where higher values indicate better performance. The commercial solver CPLEX 12.8.0 (IBM 2017) was employed to solve the RLMP and the problems (7) and (19). This specific version of CPLEX was selected to ensure a fair comparison with the leading 2BPP algorithm (Côté et al. 2021). All default settings of CPLEX were used, except for `IntParam.Threads`, which was set to one.

5.1. Results on Classic Benchmark Instances

The classic benchmark instances, generated by Berkey and Wang (1987) and Martello and Vigo (1998), are divided into 10 classes based on the method used to generate the bins and items. Each class contains 50 instances, for a total of 500 instances, with bin sizes ranging from 10×10 to 300×300 , item sizes ranging from 1×1 to 100×100 , and item counts ranging from 20 to 100.

In line with other computational studies on exact algorithms, we tested NE-BPC with a time limit of one hour and reported the number of proven optimal solutions (“#Opt”) and bins used (“#UB”) per class obtained by state-of-the-art 2BPP(-G) methods and our algorithm, as shown in Tables 3 and 4. The column “#I” indicates the number of instances, while the mark “?” denotes unreported data. A summary of the methods used in the comparison, including their short names, can be found in Table 2. It is important to note that some algorithms compute only upper bounds. To demonstrate the optimality of their solutions, following Côté et al. (2021), these were compared with the lower bounds from other literature. Specifically, for 2BPP, MV98 and BM03 are compared with the lower bounds in Monaci and Toth (2006), and PAOT10

Table 3 Number of Proven Optimal Solutions and Bins Used by Leading 2BPP Algorithms and Ours

		Ours		State-of-the-art 2BPP algorithms															
		NE-BPC		CHIB21 ²		CHIP21 ²		WOZL13 ¹		PAOT10 ¹		PS07 ²		MT06 ¹		BM03 ¹		MV98 ²	
Class	#I	#Opt	#UB	#Opt	#UB	#Opt	#UB	#Opt	#UB	#Opt	#UB	#Opt	#UB	#Opt	#UB	#Opt	#UB	#Opt	#UB
1	50	50	997	50	997	50	997	46	997	46	997	49	?	46	997	43	999	42	?
2	50	50	124	50	124	50	124	50	124	50	124	48	?	50	124	50	124	43	?
3	50	50	695	50	695	49	696	41	696	41	696	48	?	41	696	37	703	29	?
4	50	*50	119	49	120	46	123	49	120	45	123	44	?	45	124	45	125	39	?
5	50	*50	892	49	892	48	893	40	893	40	893	46	?	40	893	37	899	36	?
6	50	↑49	109	47	111	46	112	47	111	46	112	45	?	46	112	46	113	42	?
7	50	*50	825	45	826	44	827	36	827	36	828	35	?	36	827	31	832	26	?
8	50	*50	833	48	833	48	833	42	834	40	834	42	?	40	836	39	839	35	?
9	50	50	2130	50	2130	50	2130	50	2130	50	2130	50	?	50	2130	50	2130	50	?
10	50	↑46	499	41	502	40	503	38	502	36	504	28	?	36	504	32	511	27	?
Total	500	495	7223	479	7230	471	7238	439	7234	430	7241	435	?	430	7243	410	7275	369	?

¹ The number of optimal solutions is computed under the $LB = UB$ condition. Accordingly, the “#Opt” column for the heuristic methods (WOZL13, PAOT10, MT06, and BM03) does not list the values reported in the respective papers; instead, it is derived from lower bounds provided by other work.

² The “#Opt” column for the exact methods (CHIB21, CHIP21, PS07, and MV98) lists the values reported in the respective papers.

Table 4 Number of Proven Optimal Solutions and Bins Used by Leading 2BPP-G Algorithms and Ours

		Ours		State-of-the-art 2BPP-G algorithms													
		NE-BPC		GDRR22 ¹		HHTB18 ¹		ENH17 ¹		HHA14 ¹		CFIH13 ¹		CHBP11 ¹		PS07 ²	
Class	#I	#Opt	#UB	#Opt	#UB	#Opt	#UB	#Opt	#UB	#Opt	#UB	#Opt	#UB	#Opt	#UB	#Opt	#UB
1	50	50	997	50	997	50	997	50	997	46	1001	43	1004	45	1002	47	?
2	50	50	124	50	124	49	125	48	126	50	124	45	129	46	128	30	?
3	50	*50	697	47	697	47	697	39	705	38	706	33	711	29	715	44	?
4	50	↑49	120	48	121	47	122	43	126	45	124	43	126	43	126	27	?
5	50	*50	893	45	893	45	893	39	899	39	899	32	906	34	904	46	?
6	50	48	110	48	110	48	110	42	116	43	115	43	115	43	115	30	?
7	50	*50	825	37	825	36	826	28	834	32	830	24	838	27	835	34	?
8	50	*50	833	43	833	43	833	37	839	39	837	27	849	36	840	41	?
9	50	50	2130	50	2130	50	2130	50	2130	50	2130	50	2130	48	2132	50	?
10	50	↑41	503	35	503	33	505	29	509	29	509	21	517	24	514	28	?
Total	500	488	7232	453	7233	448	7238	405	7281	411	7275	361	7325	375	7311	377	?

¹ The “#Opt” column for the heuristic methods (GDRR22, HHTB18, ENH17, HHA14, CFIH13, and CHBP11) does not list the values reported in the respective papers; instead, it is derived from lower bounds provided by other work.

² The “#Opt” column for the exact method (PS07) lists the values reported in the corresponding paper.

and WOZL13 are compared with the lower bounds from the OR group Bologna website (Bologna 2019); for 2BPP-G, the lower bounds from PS07 were used to derive “#Opt” for all heuristics.

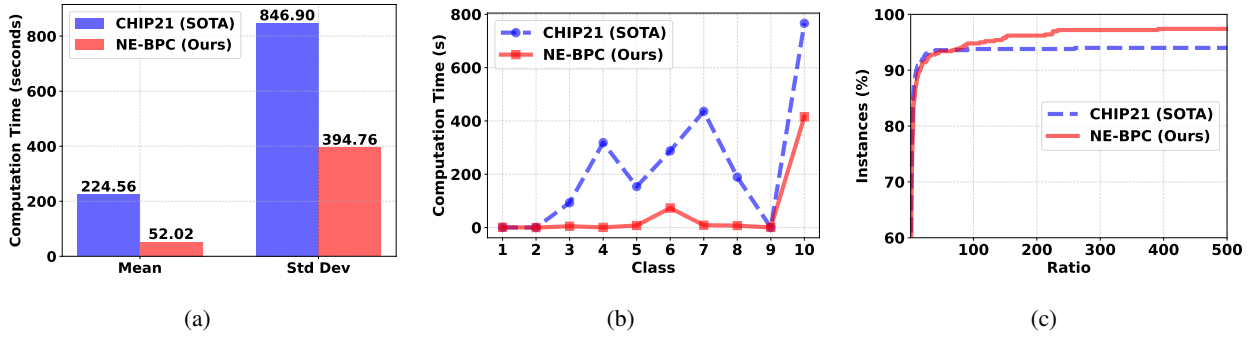
The results demonstrate that the proposed NE-BPC achieves exceptional performance on both 2BPP and 2BPP-G, solving 99% and 97.6% of instances, respectively, with a total of 7223 and 7232 bins used. Only 5 instances of the 2BPP and 12 instances of the 2BPP-G were not optimally solved by our algorithm within one hour. Moreover, NE-BPC solves all instances in several classes for the first time and either outperforms or matches previous results across all classes, as indicated by “*” and “↑” in the tables.

Comparison with State-of-the-Art Exact Algorithms. To further demonstrate the effectiveness of our algorithm, we compare NE-BPC with CHIP21 and PS07, the state-of-the-art stand-alone exact algorithms for solving 2BPP and 2BPP-G, tested on a computer with a CPU having single-thread PassMark scores of 2103 and 528 (<https://www.cpubenchmark.net>, 2025), respectively.

Table 5 reports a comparison between NE-BPC and CHIP21, where the column “#LB” indicates the sum of the best lower bounds per class. The columns “Time” and “Time_b” show the average computation time in seconds for all instances (both solved and unsolved) and the average computation time for instances solved

Table 5 Comparison with the State-of-the-Art Stand-Alone Exact 2BPP Algorithm (CHIP21) and Our NE-BPC

Class	NE-BPC (Ours)					CHIP21 (SOTA)					Advancement				
	#LB	#UB	#Opt	Time	Time _b	#LB	#UB	#Opt	Time	Time _b	Δ#LB	Δ#UB	Δ#Opt	ΔTime	ΔTime _b
1	997	997	50	1.09	1.09	997	997	50	0.33	0.33	0	0	0	225.86%	225.86%
2	124	124	50	0.07	0.07	124	124	50	0.27	0.27	0	0	0	-73.87%	-73.87%
3	695	695	50	4.94	4.77	695	696	49	93.08	21.67	0	-1	1	-94.69%	-77.98%
4	119	119	50	0.74	0.21	119	123	46	318.38	52.42	0	-4	4	-99.77%	-99.60%
5	892	892	50	7.74	6.57	891	893	48	153.77	10.27	1	-1	2	-94.97%	-36.02%
6	108	109	49	73.43	0.63	108	112	46	287.88	0.65	0	-3	3	-74.49%	-3.42%
7	825	825	50	8.90	6.02	821	827	44	435.88	5.07	4	-2	6	-97.96%	18.85%
8	833	833	50	7.30	6.28	831	833	48	189.40	47.23	2	0	2	-96.15%	-86.70%
9	2130	2130	50	0.38	0.38	2130	2130	50	0.23	0.23	0	0	0	68.69%	68.69%
10	495	499	46	415.62	11.14	493	503	40	766.37	24.53	2	-4	6	-45.77%	-54.60%
Overall	7218	7223	495	52.02	3.54	7209	7238	471	224.56	15.92	9	-15	24	-76.83%	-77.75%

**Figure 2 Bar-and-Line Chart Comparison of CHIP21 (SOTA for 2BPP) and NE-BPC (Ours)**

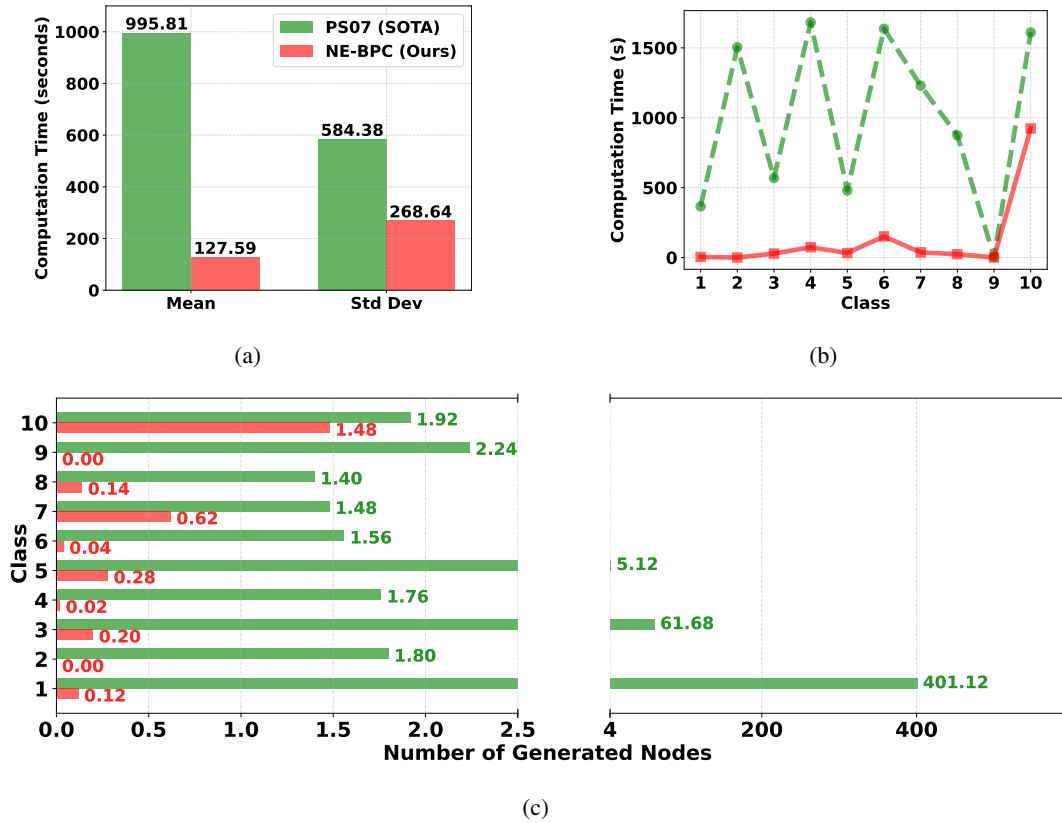
by both algorithms, respectively. To account for CPU differences, we limit NE-BPC to run for a maximum of $\lfloor 3600 \times \frac{2103}{2541} \rfloor = 2979$ seconds per instance. The data for “Time” and “Time_b” under the column “NE-BPC (Ours)” is further scaled by a factor of $\frac{2541}{2103}$. Additionally, both NE-BPC and CHIP21 use the same version of CPLEX, enhancing fairness in the comparison. The results indicate that our algorithm is highly competitive with the most advanced stand-alone exact 2BPP algorithm, obtaining 24 additional optimal solutions and requiring approximately 77% less average computation time.

Figures 2(a) and 2(b) present the bar chart and line chart, respectively, to visually demonstrate the differences in computation times between the two algorithms. Additionally, we use the performance profile proposed by Dolan and Moré (2002) to show the cumulative distribution of computation times, each normalized by the fewest computation time achieved by NE-BPC and CHIP21, as shown in Figure 2(c), which indicates that our algorithm is more effective at solving computationally expensive instances.

Table 6 reports a comparison between NE-BPC and PS07, where the columns “Cols,” “Nodes,” and “Time_r” represent the average number of patterns and nodes generated, and the average computation time in seconds for the root node, respectively. To account for CPU differences, we limit NE-BPC to a maximum runtime of $\lfloor 3600 \times \frac{528}{2541} \rfloor = 748$ seconds per instance. The data for “Time” and “Time_r” under the column “NE-BPC (Ours)” is further scaled by a factor of $\frac{2541}{528}$. Both algorithms are based on column generation and employ the RF branching scheme (Ryan and Foster 1981). The results show that our algorithm significantly outperforms the most advanced stand-alone exact 2BPP-G algorithm, achieving 109 additional

Table 6 Comparison with the State-of-the-Art Stand-Alone Exact 2BPP-G Algorithm (PS07) and Our NE-BPC

Class	NE-BPC (Ours)					PS07 (SOTA)					Advancement		
	#Opt	Cols	Nodes	Time _r	Time	#Opt	Cols	Nodes	Time _r	Time	Δ#Opt	ΔTime _r	ΔTime
1	50	2742.44	0.12	0.30	4.31	47	441.82	401.12	208.34	366.52	3	-99.85%	-98.83%
2	50	0.00	0.00	0.00	0.30	30	914.96	1.80	1457.00	1504.78	20	-100.00%	-99.98%
3	50	16579.20	0.20	16.18	28.53	44	602.56	61.68	270.28	568.98	6	-94.01%	-94.99%
4	49	3894.50	0.02	70.38	74.00	27	1115.8	1.76	1549.34	1682.32	22	-95.46%	-95.60%
5	50	16591.20	0.28	8.91	32.66	46	430.38	5.12	356.76	480.04	4	-97.50%	-93.20%
6	48	2066.08	0.04	141.51	151.24	30	1107.42	1.56	1478.50	1636.92	18	-90.43%	-90.76%
7	50	38168.24	0.62	12.56	36.84	34	407.5	1.48	1228.10	1229.64	16	-98.98%	-97.00%
8	50	26392.76	0.14	11.10	24.00	41	372.28	1.40	870.20	873.60	9	-98.72%	-97.25%
9	50	0.00	0.00	0.00	0.80	50	130.84	2.24	3.70	4.84	0	-100.00%	-83.40%
10	39	72802.52	1.48	791.37	923.18	28	1066.44	1.92	1551.62	1610.46	11	-49.00%	-42.68%
Overall	486	17923.69	0.29	105.23	127.59	377	659.00	48.01	897.38	995.81	109	-88.27%	-87.19%

**Figure 3** Bar-and-Line Chart Comparison of PS07 (SOTA for 2BPP-G) and NE-BPC

optimal solutions and requiring approximately 87% less average computation time. Moreover, NE-BPC also achieves an 88% reduction in average computation time for the root node. This improvement may be due to the generation of numerous columns, with an average of about 27 times more than PS07. We also observe that our algorithm generates only 0.29 nodes on average, which is 0.6% of the number generated by PS07. This can be attributed to primal heuristics and enhanced dual bounds through additional SR and VB cuts. Figures 3(a)-3(c) respectively present two bar charts and a line chart that visually demonstrate the differences between the two algorithms in computation time and the number of generated nodes.

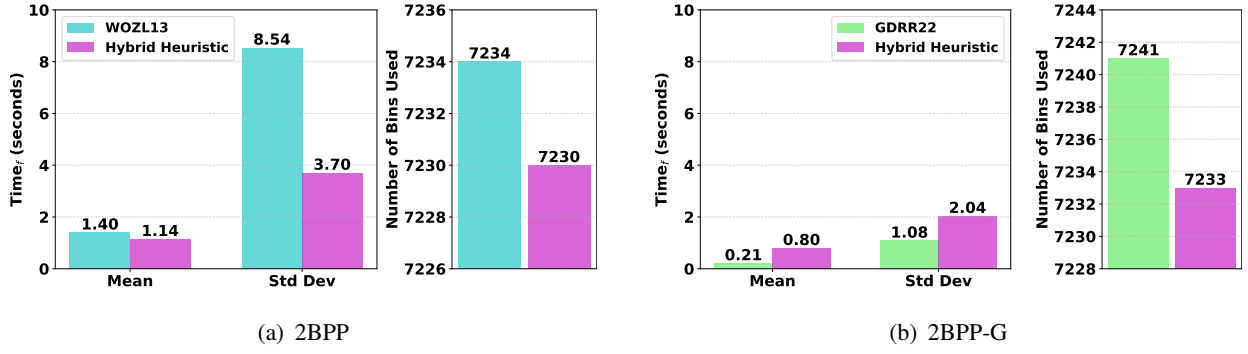


Figure 4 Bar Chart Comparison of State-of-the-Art 2BPP(-G) Heuristics and Our Hybrid Heuristic

Comparison with State-of-the-Art Heuristic Algorithms. We compare our hybrid heuristic with the leading heuristics (WOZL13 and GDRR22) for solving 2BPP and 2BPP-G, respectively.

Figure 4(a) presents two bar charts that report the average time (“Time_f”) required to obtain the best solution and the total number of bins used by WOZL13 and our hybrid heuristic. Because WOZL13 was tested on a computer whose CPU has a single-thread PassMark score of 1161, we scale our times by the factor $\frac{2541}{1161}$. A similar comparison between GDRR22 and our hybrid heuristic is shown in Figure 4(b). Because the original output of GDRR22 does not include “Time_f”, we modified the [GDRR22 code](#) shared by [Gardeyn and Wauters \(2022\)](#) and executed it on our computer. Consistent with our hybrid heuristic (see §EC.2.1 of the E-Companion), GDRR22 was limited to run for a maximum of $\frac{n}{8} + 11.5$ seconds, whereas WOZL13 was limited to 120 seconds. The results show that the proposed hybrid heuristic obtains very high-quality solutions with an average “Time_f” of about one second for both problems. Although the average time our heuristic requires to find the best solution is only slightly shorter than that of WOZL13 and longer than that of GDRR22, it uses fewer bins under identical or similar time limits and is therefore highly competitive with the leading heuristics. Note that heuristic solutions for 2BPP(-G) have been extensively investigated for decades; hence, even minor improvements are challenging. For example, WOZL13 reduces bin usage by only two relative to its previous best-performing heuristic. Consequently, the improvements achieved by our hybrid heuristic represent a meaningful contribution to the development of related heuristic methods.

5.2. Results on New Benchmark Instances

The new benchmark instances, generated by [Wang et al. \(2025a\)](#) (see Section B of the Appendix of their paper), are divided into 10 more challenging classes based on the method used to generate the bins and items. Each class contains 50 instances, for a total of 500 instances, with bin sizes ranging from 20×20 to 600×600 , item sizes ranging from 1×1 to 150×150 , and item counts ranging from 100 to 500.

Tables 7 and 8 report the 2BPP and 2BPP-G results of NE-BPC on these new benchmark instances. The columns “#Opt₀,” “#I_r,” “Gap,” “#LB₀,” “#UB₀,” “Gap₀,” “Cuts,” and “Time₀” represent, respectively, the number of optimal solutions proven by the initial bounds, the number of unsolved instances with incomplete CRG at the root node, the percentage optimality gap ($100 \times \frac{UB-LB}{UB}$), the sum of the initial lower and

Table 7 2BPP Results of NE-BPC on New Benchmark Instances

Class	#I	#Opt	#Opt ₀	#I _r ^u	#LB	#UB	Gap	#LB ₀	#UB ₀	Gap ₀	Cols	Cuts	Nodes	Time ₀	Time _r	Time
11	50	50	49	0	2429	2429	0.00	2429	2430	0.13	5348.0	677.8	5.7	0.84	0.39	20.89
12	50	50	50	0	291	291	0.00	291	291	0.00	0.0	0.0	0.0	0.49	0.00	0.53
13	50	43	42	6	1554	1561	0.41	1554	1562	0.58	103291.3	419.4	0.9	10.14	472.92	511.38
14	50	49	48	1	267	268	0.50	267	269	0.83	7567.9	0.0	0.0	5.69	73.17	78.91
15	50	17	8	18	2170	2221	2.12	2170	2245	3.26	416023.4	9158.3	27.3	45.88	1723.19	2599.09
16	50	48	47	2	262	264	0.72	262	265	1.22	12016.8	0.0	0.1	54.28	181.57	235.96
17	50	49	48	0	2565	2566	0.02	2564	2566	0.09	20180.2	360.0	0.4	8.33	23.87	83.97
18	50	49	47	1	2570	2571	0.04	2569	2572	0.12	31027.6	115.0	0.1	8.37	96.78	106.47
19	50	50	32	0	4989	4989	0.00	4985	5007	0.35	28448.5	13.4	0.5	27.46	2.08	29.84
20	50	40	27	2	1230	1240	0.80	1230	1253	1.79	348550.6	4707.1	8.0	30.27	272.77	914.13
Overall	500	445	398	30	18327	18400	0.46	18321	18460	0.84	97245.4	1545.1	4.3	19.18	284.68	458.12

Table 8 2BPP-G Results of NE-BPC on New Benchmark Instances

Class	#I	#Opt	#Opt ₀	#I _r ^u	#LB	#UB	Gap	#LB ₀	#UB ₀	Gap ₀	Cols	Cuts	Nodes	Time ₀	Time _r	Time
11	50	49	49	0	2429	2430	0.13	2429	2430	0.13	12732.3	534.3	4.3	1.02	2.03	72.62
12	50	50	50	0	291	291	0.00	291	291	0.00	0.0	0.0	0.0	0.14	0.00	0.18
13	50	43	42	7	1554	1561	0.41	1554	1562	0.58	104603.1	344.5	1.4	9.82	501.42	552.50
14	50	47	47	3	267	270	1.17	267	270	1.17	7493.5	0.0	0.1	6.92	212.09	219.08
15	50	11	7	31	2170	2236	2.89	2170	2270	4.09	314828.9	2540.3	6.0	42.87	2515.82	2889.92
16	50	47	47	3	262	265	1.22	262	265	1.22	4907.1	0.0	0.1	133.04	188.03	321.21
17	50	48	43	2	2564	2566	0.09	2564	2571	0.29	79702.4	1082.4	1.3	12.67	152.81	198.25
18	50	47	45	2	2569	2572	0.12	2569	2574	0.18	60548.2	165.9	0.2	11.52	212.54	228.35
19	50	50	26	0	4990	4990	0.00	4985	5017	0.50	32188.9	98.5	0.7	31.59	3.63	35.50
20	50	30	22	8	1230	1250	1.54	1230	1258	2.07	320163.6	3447.6	5.4	36.92	757.67	1522.75
Overall	500	422	378	56	18326	18431	0.76	18321	18508	1.02	93716.8	821.4	1.9	28.65	454.61	604.04

upper bounds, the average percentage initial optimality gap ($100 \times \frac{UB_0 - LB_0}{UB_0}$), the average number of generated SR cuts, and the average time (in seconds) for computing the initial bounds. The time limit is one hour. Overall, NE-BPC still shows high efficiency on these larger-scale instances, solving 89% and 84% of the instances for 2BPP and 2BPP-G, respectively. The final solutions use 60 and 77 fewer bins than the initial solutions, respectively, demonstrating the powerful performance of our algorithm in searching for upper bounds. Moreover, the hybrid heuristic provides high-quality initial solutions for NE-BPC, achieving 0.84% and 1.02% initial optimality gaps within 20 and 29 seconds on average, respectively. However, these instances pose significant challenges at the root node, which consumes most of the computation time. Of the 55 and 78 unsolved instances, over 55% and 72% were unable to complete the CRG at the root node to obtain a tighter lower bound. We also observed that the largest absolute gap between UB and LB is as high as 3. Therefore, these instances present a new and highly challenging benchmark for future research.

5.3. Analysis of the Effectiveness of the Key Techniques of NE-BPC

Tables 9 and 10 present the differences between the 2BPP and 2BPP-G results for several versions of NE-BPC and the full version (which includes all components), respectively. Specifically, “NoHeuPricing” refers to NE-BPC being run without heuristic pricing. “NoRoundingHeu” refers to NE-BPC being run without the rounding heuristic. “NoDivingHeu” refers to NE-BPC being run without the diving heuristic. “NoHybridHeu” refers to NE-BPC being run without the hybrid heuristic. “NoRotateIns” refers to NE-BPC being run without rotating instances. “NoSRCuts” and “NoVBCut” refer to NE-BPC being run without considering SR cuts (3) and the VB cut (4), respectively.

Table 9 Differences between the 2BPP Results for Several Versions of NE-BPC and the Full Version

Class	NoHeuPricing				NoHybridHeu				NoSRCuts				NoRoundingHeu				NoRotateIns				NoDivingHeu				NoVBCut			
	$\Delta\#Opt$	$\Delta\#UB$	$\Delta\#Time$	$\Delta\#Opt/\Delta\#UB$	$\Delta\#Time$	$\Delta\#Opt/\Delta\#UB$	$\Delta\#Time$	$\Delta\#Opt/\Delta\#UB$	$\Delta\#Time$	$\Delta\#Opt/\Delta\#UB$	$\Delta\#Time$	$\Delta\#Opt/\Delta\#UB$	$\Delta\#Time$	$\Delta\#Opt/\Delta\#UB$	$\Delta\#Time$	$\Delta\#Opt/\Delta\#UB$	$\Delta\#Time$	$\Delta\#Opt/\Delta\#UB$	$\Delta\#Time$	$\Delta\#Opt/\Delta\#UB$	$\Delta\#Time$	$\Delta\#Opt/\Delta\#UB$	$\Delta\#Time$	$\Delta\#Opt/\Delta\#UB$				
1	0	0	3.1%	0	0	-35.5%	0	0	-1.0%	0	-2.1%	0	0	-3.6%	0	-1.0%	0	-1.0%	0	-1.0%	0	-1.0%	0	-1.0%				
2	0	0	-6.1%	0	0	515.9%	0	0	-5.3%	0	0	0	0	-2.7%	0	-2.7%	0	-2.1%	0	-2.1%	0	-2.1%	0	-5.5%				
3	0	0	1292.1%	0	0	4.4%	0	0	-4.6%	0	0	0	0	-6.7%	0	-6.7%	0	-2.0%	0	-2.0%	0	-2.0%	0	3.8%				
4	0	0	1.2%	-1	1	12650.9%	0	0	0.5%	0	0	0	0	2.0%	-1	1	11640.7%	0	0	0	0	0	0	1.5%				
5	0	0	145.8%	0	0	-52.3%	0	0	-0.4%	0	-2.9%	0	0	-2.8%	0	-2.8%	0	1.0%	0	1.0%	0	1.0%	0	2.5%				
6	0	0	0.0%	-1	1	98.2%	0	0	0.0%	0	0	0	0	0.0%	0	0.2%	0	0	0	-0.1%	0	0	0	0.0%				
7	-1	1	1430.7%	0	0	-55.3%	0	0	-4.9%	0	2.8%	0	0	12.6%	0	12.6%	0	16.7%	0	16.7%	0	16.7%	0	-9.5%				
8	0	0	1696.8%	0	0	-38.3%	-1	0	1187.5%	0	4.6%	0	0	10.5%	0	10.5%	0	0.8%	0	0.8%	0	0.8%	0	-23.2%				
9	0	0	-0.1%	0	0	55.1%	0	0	-3.7%	0	-1.0%	0	0	-0.8%	0	-0.8%	0	0	0	0	0	0	0	-1.5%				
10	-8	2	136.0%	-1	0	1.9%	-1	1	17.8%	-2	1	24.5%	0	3.9%	-1	1	21.9%	-1	1	21.9%	-1	1	21.9%	0	3.7%			
Overall	-9	3	163.5%	-3	2	30.1%	-2	1	28.7%	-2	1	19.8%	-1	1	17.9%	-1	1	17.8%	-1	1	17.8%	-1	1	17.8%	0	2.6%		

Table 10 Differences between the 2BPP-G Results for Several Versions of NE-BPC and the Full Version

Class	NoHybridHeu				NoHeuPricing				NoSRCuts				NoRotateIns				NoRoundingHeu				NoVBCut				NoDivingHeu			
	$\Delta\#Opt$	$\Delta\#UB$	$\Delta\#Time$	$\Delta\#UB$	$\Delta\#Opt$	$\Delta\#UB$	$\Delta\#Time$	$\Delta\#UB$	$\Delta\#Opt$	$\Delta\#UB$	$\Delta\#Time$	$\Delta\#UB$	$\Delta\#Opt$	$\Delta\#UB$	$\Delta\#Time$	$\Delta\#UB$	$\Delta\#Opt$	$\Delta\#UB$	$\Delta\#Opt$	$\Delta\#UB$	$\Delta\#Time$	$\Delta\#UB$	$\Delta\#Opt$	$\Delta\#UB$	$\Delta\#Time$	$\Delta\#UB$		
1	0	0	-31.8%	0	0	9.7%	0	0	-0.3%	0	0.7%	0	0	-1.1%	0	-1.1%	0	-1.1%	0	-1.1%	0	-1.1%	0	-1.1%	0	-0.5%	0	
2	-1	1	115668.7%	0	0	-1.7%	0	0	0.7%	0	14.7%	0	0	0.0%	0	0.0%	0	0.0%	0	0.0%	0	0.1%	0	0.1%	0	0.4%	0	
3	0	0	42.0%	0	0	967.8%	0	0	-3.2%	0	0	0	0	-3.4%	0	-3.4%	0	-3.4%	0	-3.4%	0	-3.8%	0	-3.8%	0	0.4%	0	
4	-4	4	398.2%	0	0	0.0%	0	0	0.0%	-2	2	198.8%	0	0.0%	0	0.0%	0	0.0%	0	0.0%	0	0.1%	0	0.1%	0	0.0%	0	
5	0	0	-31.7%	-1	0	1156.6%	0	0	0.0%	0	5.0%	0	0	8.8%	0	8.8%	0	8.8%	0	8.8%	0	3.5%	0	3.5%	0	-0.8%	0	
6	-4	4	199.9%	0	0	0.0%	0	0	0.0%	0	0.2%	0	0	0.0%	0	0.0%	0	0.0%	0	0.0%	0	0.0%	0	0.0%	0	0.0%	0	
7	0	1	-46.6%	-1	0	1331.6%	-1	1	939.2%	0	25.4%	0	0	59.7%	0	59.7%	0	59.7%	0	59.7%	0	-6.9%	0	-6.9%	0	1.2%	0	
8	0	0	-10.3%	-1	0	3245.8%	-1	0	1437.2%	0	12.3%	0	0	63.7%	0	63.7%	0	63.7%	0	63.7%	0	10.9%	0	10.9%	0	0.3%	0	
9	0	0	94.7%	0	0	-3.5%	0	0	-5.3%	0	82.9%	0	0	-2.8%	0	-2.8%	0	-2.8%	0	-2.8%	0	-0.4%	0	-0.4%	0	-0.6%	0	
10	-1	0	9.0%	-5	0	33.7%	-1	0	2.1%	1	-11.3%	0	0	-0.9%	0	-0.9%	0	-0.9%	0	-0.9%	0	-0.9%	0	-0.9%	0	-1.0%	0	
Overall	-10	10	71.6%	-8	0	65.7%	-3	1	15.9%	-1	1	6.2%	0	0	0.1%	0	0.1%	0	0.1%	0	0	-0.7%	0	-0.7%	0	-0.8%	0	

Table 11 Comparison of NE-BPC with and without Heuristics

Class	#I	FullVersion				NoHeus				2BPP				FullVersion				2BPP-G				FullVersion				NoHeus			
		#Opt	#L _{ru}	Gap	Time	#Opt	#L _{ru}	Gap	Time	#Opt	#L _{ru}	Gap	Time	#Opt	#L _{ru}	Gap	Time	#Opt	#L _{ru}	Gap	Time	#Opt	#L _{ru}	Gap	Time	#Opt	#L _{ru}	Gap	Time
		#I	#L _{ru}	Gap	Time	#I	#L _{ru}	Gap	Time	#I	#L _{ru}	Gap	Time	#I	#L _{ru}	Gap	Time	#I	#L _{ru}	Gap	Time	#I	#L _{ru}	Gap	Time	#I	#L _{ru}	Gap	Time
1	50	50	0	0.00	0.90	50	0	0.00	4.29	50	0	0.00	0.89	50	0	0.00	0.89	50	0	0.00	0.89	50	0	0.00	0.89	50	0	0.00	9.11
2	50	50	0	0.00	0.06	42	8	6.67	655.70	50	0	0.00	0.06	50	0	0.00	0.06	50	0	0.00	0.06	50	0	0.00	0.06	50	0	0.00	1670.21
3	50	50	0	0.00	4.09	43	7	9.93	715.13	50	0	0.00	6.11	50	0	0.00	6.11	50	0	0.00	6.11	50	0	0.00	6.11	50	0	0.00	2565.94
4	50	50	0	0.00	0.62	26	24	43.87	1755.84	50	0	0.00	72.41	50	0	0.00	72.41	50	0	0.00	72.41	50	0	0.00	72.41	50	0	0.00	2592.07
5	50	50	0	0.00	6.40	32	18	25.66	1484.20	50	0	0.00	6.89	50	0	0.00	6.89	50	0	0.00	6.89	50	0	0.00	6.89	50	0	0.00	2601.20
6	50	49	1	1.00	73.19	14	36	69.41	2592.10	50	0	0.00	145.53	50	0	0.00	145.53	50	0	0.00	145.53	50	0	0.00	145.53	50	0	0.00	2736.06
7	50	50	0	0.00	7.36	43	7	10.08	705.54	50	0	0.00	7.60	50	0	0.00	7.60	50	0	0.00	7.60	50	0	0.00	7.60	50	0	0.00	1294.80
8	50	50	0	0.00	6.04	41	9	11.43	736.20	50	0	0.00	4.99	50	0	0.00	4.99	50	0	0.00	4.99	50	0	0.00	4.99	50	0	0.00	1209.53
9	50	50	0	0.00	0.32	50	0	0.00	30.67	50	0	0.00	0.17	50	0	0.00	0.17	50	0	0.00	0.17	50	0	0.00	0.17	50	0	0.00	438.78
10	50	46	2	0.80	393.63	18	31	45.08	2447.87	50	0	0.00	754.69	50	0	0.00	754.69	50	0	0.00	754.69	50	0	0.00	754.69	50	0	0.00	3225.48
Overall	500	495	3	0.18	49.26	359	140	22.21	1112.75	488	10	0.40	99.94	488	10	0.40	99.94	488	10	0.40	99.94	488	10	0.40	99.94	488	10	0.40	1834.32

The results demonstrate that heuristic pricing, the hybrid heuristic, and SR cuts are critical to NE-BPC and significantly improve its performance. Specifically, heuristic pricing significantly speeds up the CRG process; without it, the CRG for the root node failed to complete for 9 instances. The hybrid heuristic finds several solutions that subsequent BPC enumeration cannot find. Without SR cuts, two instances encountered memory overflows (when solving 2BPP-G) due to huge enumeration trees. This highlights that SR cuts effectively improve the LP relaxation at each node, thereby strengthening pruning and reducing the enumeration tree size. Other components provide modest performance improvements.

The rounding heuristic improves the quality of primal solutions and accelerates the CRG, leading to a tighter lower bound and enabling the solution of one additional instance. The instance rotation strategy allows several instances, which are unsolvable by BPC enumeration, to be solved using initial bounds. The diving heuristic identifies one additional optimal solution, while the VB cut facilitates the solution of another instance by computing a tighter lower bound.

Furthermore, to analyze the contribution of the embedded heuristics to the performance of the proposed algorithm, we also test a specific version of NE-BPC, referred to as "NoHeus," in which the hybrid, rounding, diving, and pricing heuristics are disabled. Its results are compared with those of the full version of NE-BPC in Table 11, which shows that without these heuristics, the algorithm becomes extremely inefficient. In hundreds of instances, the CRG of the root node cannot be completed, resulting in a significant increase in average computation time, approximately 23 and 18 times for 2BPP and 2BPP-G respectively. This clearly demonstrates that the heuristics we introduced can significantly enhance NE-BPC.

5.4. Analysis of the Non-Numerically Exact Version of NE-BPC

A non-numerically exact version of NE-BPC, denoted BPC, was implemented to test whether numerical-safety issues arise when computing unsafe bounds. In this version, the pricing algorithm uses the original floating-point dual solutions instead of the scaled integer dual solutions. In addition, the optimal value of $G2KP(\mu, \lambda)$ restricted to $[z_1, z_2]$ —that is, $z_{G2KP}^*(\mu, \lambda, z_1, z_2)$ —is computed by the non-numerically exact algorithm of Wang et al. (2025a).

Table 12 compares the results of BPC and NE-BPC. Overall, BPC performs worse than NE-BPC, which suggests that the numerically exact pricing algorithm of Wang et al. (2025a) not only avoids numerical issues but also enhances computing performance. It is worth noting that, despite BPC's weaker overall performance, we observe no obvious errors; for example, the lower bounds proved by BPC are never larger than the optimal solution values proved by NE-BPC. We only note that BPC computes a better lower bound for one of the 2000 (2BPP and 2BPP-G, classic and new) instances unsolved by NE-BPC; the "improvement" may result from numerical inaccuracies. In particular, for 2BPP, BPC optimally solves the 12th instance in Class 6 by proving a tighter lower bound and the 32nd instance in Class 10 by finding a better primal solution, which neither NE-BPC nor leading methods in the literature can solve within one hour. Below, we analyze the reasons for obtaining the improved lower bound to identify potential errors.

Table 12 Difference of the Results of BPC and NE-BPC

Classic instances							New instances						
n	2BPP			2BPP-G			n	2BPP			2BPP-G		
	$\Delta\#Opt$	$\Delta\#LB$	$\Delta Time$	$\Delta\#Opt$	$\Delta\#LB$	$\Delta Time$		$\Delta\#Opt$	$\Delta\#LB$	$\Delta Time$	$\Delta\#Opt$	$\Delta\#LB$	$\Delta Time$
20	0	0	13.6%	0	0	11.4%	100	0	0	-4.1%	-1	0	5.1%
40	0	0	1.4%	-2	-2	105.7%	200	-6	-2	38.9%	1	0	3.7%
60	0	0	6.3%	-1	-1	34.8%	300	-1	0	8.2%	-1	0	5.8%
80	0	-1	31.0%	-1	-2	48.1%	400	-1	0	7.5%	-1	0	4.9%
100	-5	-4	322.5%	-4	-5	108.1%	500	-1	0	3.2%	-1	0	4.6%
Overall	-5	-5	90.9%	-8	-10	73.5%	Overall	-9	-2	10.2%	-3	0	4.8%

Table 13 Results of Solving Five 2BPP Instances with a Time Limit of 8 Hours (28800 Seconds)

Instance	n	W	H	Initial bounds		NE-BPC					BPC				
				LB_0	UB_0	LB	UB	Opt	Nodes	Time	LB	UB	Opt	Nodes	Time
Class-06-12	40	300	300	1	2	1	2		1	T.L.	2	2	✓	1	66.71
Class-10-24	60	100	100	7	8	7	8		1	T.L.	—	—	—	—	—
Class-10-29	60	100	100	8	9	8	9		1	T.L.	9	9	✓	1	24150.06
Class-10-32	80	100	100	10	11	10	10	✓	188	5232.60	10	10	✓	19	359.24
Class-10-47	100	100	100	13	14	13	14		134	T.L.	13	14		122	T.L.

“T.L.” indicates that the time limit has been reached, and “—” indicates that a memory overflow has occurred.

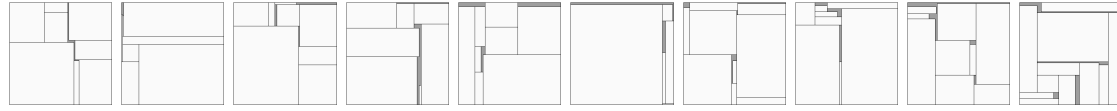
Specifically, we observed that, in the fifteenth CRG iteration at the root node, the non-numerically exact pricing algorithm (N-NEA) reported a minimal reduced cost of $\delta_1^* = -0.03649635036496335$, which triggered the fathoming rule. In contrast, the numerically exact pricing algorithm confirmed a minimal reduced cost of $\delta_2^* = -0.04379562019$. Further analysis showed that this discrepancy arose from the bounding cut introduced by Wang et al. (2025a). In this CRG iteration, the lower bound on the reduced cost was computed as $\underline{\delta} = -0.04379562043795589$. To accelerate the model-based N-NEA, the bounding cut was enforced, requiring the reduced cost to be no less than $\underline{\delta}$. Because δ_2^* and $\underline{\delta}$ are nearly identical, CPLEX can incorrectly conclude that the solution with cost δ_2^* violates the cut, thereby returning a suboptimal solution with cost δ_1^* . Although removing the cut or tightening the tolerance from the default 10^{-6} to 10^{-9} allows N-NEA to recover the optimal solution, other numerical errors remain unavoidable and unpredictable. Consequently, numerically exact implementations are essential to guarantee solution optimality.

5.5. Additional Experiments for Unsolved Classic 2BPP Instances

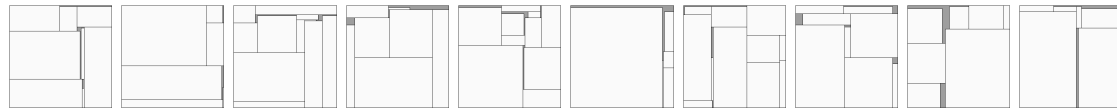
Let “Class-c-d” denote the d-th instance in Class c. For 2BPP, Class-06-12, Class-10-24, Class-10-29, Class-10-32, and Class-10-47 cannot be solved by NE-BPC within a one-hour time limit. Except for Class-10-24, which has been proven by Côté et al. (2021) to have an optimal solution value of 8, the other instances have remained unsolved for decades. To further investigate, we extended the time limit to 8 hours and ran both NE-BPC and BPC on these instances, with the results shown in Table 13. Both algorithms solve Class-10-32 by finding a better primal solution. Illustrations of the solutions found are provided in Figure 5. In addition, BPC solves Class-06-12 and Class-10-29 by improving the lower bound. However, as discussed earlier, the bound may be incorrect due to numerical errors.



(a) Initial Solution Found by the Hybrid Heuristic Using 11 Bins



(b) Optimal Solution Found by NE-BPC (with a Time Limit of 8 Hours) Using 10 Bins



(c) Optimal Solution Found by BPC Using 10 Bins

Figure 5 Illustrations of the Three Solutions Found by Our Algorithms for the Class-10-32 Instance

6. Concluding Remarks

This paper examined the classic 2BPP and its prominent variant (2BPP-G), both of which are fundamental problems in combinatorial optimization that have been extensively studied over the years. We introduced a unified, highly efficient, and numerically exact BPC algorithm to solve these problems, integrating our novel ideas with advanced techniques from existing research. In particular, the newly developed heuristic techniques are highly competitive with leading alternatives and significantly improve the algorithm's performance, setting our BPC method apart from prior approaches due to its superior efficiency and numerical safety. Thus, we believe this work represents a substantial advancement in solving 2BPPs, and we have made our state-of-the-art code publicly available to encourage further progress in related research areas.

The proposed algorithm (NE-BPC) was exhaustively tested on the classic benchmark instances from [Berkey and Wang \(1987\)](#) and [Martello and Vigo \(1998\)](#), as well as on the new benchmark instances from [Wang et al. \(2025a\)](#), comprising a total of 1000 instances. The bin sizes ranged from 10×10 to 600×600 , item sizes ranged from 1×1 to 150×150 , and item counts ranged from 20 to 500. The primary results are summarized in Table 14. NE-BPC solved nearly all classic instances and solved 83% and 79% of the large-scale instances containing 500 items, with an average optimality gap of only 0.51% and 0.63% for 2BPP and 2BPP-G, respectively, marking significant progress. Furthermore, NE-BPC demonstrated substantial improvements over state-of-the-art methods ([Côté et al. 2021](#), [Pisinger and Sigurd 2007](#)), obtaining 24 and 109 additional optimal solutions and reducing computation time by 77% and 87%, respectively. Numerous classic instances, unsolved for more than three decades, were finally solved to optimality.

These results highlight the adaptability of the proposed algorithm and its potential to efficiently address other problem variants involving additional practical constraints, such as precedence constraints ([Turbian](#)

Table 14 Summary of the 2BPP(-G) Results of NE-BPC on Classic and New Benchmark Instances

Classic benchmark instances									New benchmark instances								
n	#I	2BPP			2BPP-G				n	#I	2BPP			2BPP-G			
		#Opt	Gap	Time	#Opt	Gap	Time				#Opt	Gap	Time	#Opt	Gap	Time	
20	100	100	0.00%	0.30	100	0.00%	0.32		100	100	95	0.34%	213.36	93	0.47%	277.47	
40	100	99	0.50%	37.68	98	1.00%	73.95		200	100	93	0.71%	355.55	85	1.29%	600.39	
60	100	98	0.24%	74.20	98	0.24%	101.88		300	100	87	0.40%	558.14	81	0.92%	721.71	
80	100	99	0.09%	73.37	96	0.49%	160.56		400	100	87	0.35%	497.42	84	0.47%	601.67	
100	100	99	0.07%	60.76	96	0.25%	162.97		500	100	83	0.51%	666.12	79	0.63%	818.93	
Overall	500	495	0.18%	49.26	488	0.40%	99.94		Overall	500	445	0.46%	458.12	422	0.76%	604.04	

et al. 2022), customer order spread (Martin et al. 2024), item conflicts and load balancing (Sun et al. 2025), circular bins and/or items (Lai et al. 2022, Zhou et al. 2025, Wang et al. 2025c), and irregular bins and/or items (Wang et al. 2022, Zhang et al. 2022a, Cai et al. 2023). In addition, it can provide useful insights into pricing algorithms and primal heuristics for designing (numerically exact) BPC methods that involve complex pricing problems, such as the BPC approaches for the classic, highly challenging Three-Dimensional Bin-Packing Problem (Lodi et al. 2002a, Crainic et al. 2008, Fontaine and Minner 2023) and its variants (Gzara et al. 2020, Lamas-Fernandez et al. 2023, Zhu et al. 2024). These represent interesting research directions and can be further effectively explored based on our comprehensive public repository.

Acknowledgments

The authors gratefully acknowledge Jean-François Côté for kindly providing the detailed 2BPP results from Côté et al. (2021), Jeroen Gardeyn for generously sharing the GDRR22 code (<https://github.com/JeroenGar/gdrr-2bp>) from Gardeyn and Wauters (2022), Mauro Russo for discussing insights into solving the G2KP with guillotine constraints, and Iori et al. (2022) for establishing the comprehensive two-dimensional packing library (2DPackLib), which supplied all the classic benchmark instances that were instrumental to this study.

References

- Alvarez-Valdés R, Parreño F, Tamarit JM (2009) A branch and bound algorithm for the strip packing problem. *OR Spectrum* 31:431–459.
- Baldacci R, Bartolini E, Mingozzi A (2011) An exact algorithm for the pickup and delivery problem with time windows. *Operations Research* 59(2):414–426.
- Baldacci R, Coniglio S, Cordeau JF, Furini F (2024) A numerically exact algorithm for the bin-packing problem. *INFORMS Journal on Computing* 36(1):141–162.
- Barnhart C, Johnson EL, Nemhauser GL, Savelsbergh MW, Vance PH (1998) Branch-and-price: Column generation for solving huge integer programs. *Operations Research* 46(3):316–329.
- Ben Amor H, Desrosiers J, Valério de Carvalho JM (2006) Dual-optimal inequalities for stabilized column generation. *Operations Research* 54(3):454–463.
- Berkey JO, Wang PY (1987) Two-dimensional finite bin-packing algorithms. *Journal of the Operational Research Society* 38(5):423–429.

- Bierwirth C, Meisel F (2010) A survey of berth allocation and quay crane scheduling problems in container terminals. *European Journal of Operational Research* 202(3):615–627.
- Blum C, Schmid V, Baumgartner L (2012) On solving the oriented two-dimensional bin packing problem under free guillotine cutting: Exploiting the power of probabilistic solution construction. *arXiv preprint arXiv:1209.0880*.
- Bologna OG (2019) Library of instances: Two-dimensional bin packing problem. <http://or.dei.unibo.it/library/two-dimensional-bin-packing-problem>, accessed August 15, 2020.
- Boschetti MA, Mingozzi A (2003a) The two-dimensional finite bin packing problem. Part I: New lower bounds for the oriented case. *Quarterly Journal of the Belgian, French and Italian Operations Research Societies* 1(1):27–42.
- Boschetti MA, Mingozzi A (2003b) The two-dimensional finite bin packing problem. Part II: New lower and upper bounds. *Quarterly Journal of the Belgian, French and Italian Operations Research Societies* 1(2):135–147.
- Boschetti MA, Mingozzi A, Hadjiconstantinou E (2002) New upper bounds for the two-dimensional orthogonal non-guillotine cutting stock problem. *IMA Journal of Management Mathematics* 13(2):95–119.
- Cai S, Deng J, Lee LH, Chew EP, Li H (2023) Heuristics for the two-dimensional irregular bin packing problem with limited rotations. *Computers & Operations Research* 160:106398.
- Carlier J, Clautiaux F, Moukrim A (2007) New reduction procedures and lower bounds for the two-dimensional bin packing problem with fixed orientation. *Computers & Operations Research* 34(8):2223–2250.
- Castro PM, Oliveira JF (2011) Scheduling inspired models for two-dimensional packing problems. *European Journal of Operational Research* 215(1):45–56.
- Charalambous C, Fleszar K (2011) A constructive bin-oriented heuristic for the two-dimensional bin packing problem with guillotine cuts. *Computers & Operations Research* 38(10):1443–1451.
- Chen CS, Lee SM, Shen Q (1995) An analytical model for the container loading problem. *European Journal of Operational Research* 80(1):68–76.
- Clautiaux F, Carlier J, Moukrim A (2007) A new exact method for the two-dimensional bin-packing problem with fixed orientation. *Operations Research Letters* 35(3):357–364.
- Contardo C, Martinelli R (2014) A new exact algorithm for the multi-depot vehicle routing problem under capacity and route length constraints. *Discrete Optimization* 12:129–146.
- Côté JF, Dell’Amico M, Iori M (2014) Combinatorial Benders’ cuts for the strip packing problem. *Operations Research* 62(3):643–661.
- Côté JF, Haouari M, Iori M (2021) Combinatorial Benders decomposition for the two-dimensional bin packing problem. *INFORMS Journal on Computing* 33(3):963–978.
- Côté JF, Iori M (2018) The meet-in-the-middle principle for cutting and packing problems. *INFORMS Journal on Computing* 30(4):646–661.
- Crainic TG, Perboli G, Tadei R (2008) Extreme point-based heuristics for three-dimensional bin packing. *INFORMS Journal on Computing* 20(3):368–384.

- Cui YP, Yao Y, Zhang D (2018) Applying triple-block patterns in solving the two-dimensional bin packing problem. *Journal of the Operational Research Society* 69(3):402–415.
- da Silva RF, Schouery R (2023) A branch-and-cut-and-price algorithm for cutting stock and related problems. *arXiv preprint arXiv:2308.03595*.
- de Lima VL, Iori M, Miyazawa FK (2023) Exact solution of network flow models with strong relaxations. *Mathematical Programming* 197(2):813–846.
- Delorme M, Iori M (2020) Enhanced pseudo-polynomial formulations for bin packing and cutting stock problems. *INFORMS Journal on Computing* 32(1):101–119.
- Desrosiers J, Lübbecke M, Desaulniers G, Gauthier J (2024) Branch-and-price. *Les Cahiers du GERAD* ISSN 711:2440.
- Dolan ED, Moré JJ (2002) Benchmarking optimization software with performance profiles. *Mathematical Programming* 91(2):201–213.
- Farley AA (1990) A note on bounding a class of linear programming problems, including cutting stock problems. *Operations Research* 38(5):922–923.
- Faroe O, Pisinger D, Zachariasen M (2003) Guided local search for the three-dimensional bin-packing problem. *INFORMS Journal on Computing* 15(3):267–283.
- Firat H, Alpaslan N (2020) An effective approach to the two-dimensional rectangular packing problem in the manufacturing industry. *Computers & Industrial Engineering* 148:106687.
- Fleszar K (2013) Three insertion heuristics and a justification improvement heuristic for two-dimensional bin packing with guillotine cuts. *Computers & Operations Research* 40(1):463–474.
- Fontaine P, Minner S (2023) A branch-and-repair method for three-dimensional bin selection and packing in e-commerce. *Operations Research* 71(1):273–288.
- Francescatto M, Júnior AN (2025) Two-dimensional bin packing, cutting stock, and open dimension problems: A survey of practical requirements. *Computers & Operations Research* 107209.
- Furini F, Malaguti E (2013) Models for the two-dimensional two-stage cutting stock problem with multiple stock size. *Computers & Operations Research* 40(8):1953–1962.
- Furini F, Malaguti E, Thomopulos D (2016) Modeling two-dimensional guillotine cutting problems via integer programming. *INFORMS Journal on Computing* 28(4):736–751.
- Gardeyn J, Wauters T (2022) A goal-driven ruin and recreate heuristic for the 2D variable-sized bin packing problem with guillotine constraints. *European Journal of Operational Research* 301(2):432–444.
- Garey MR, Johnson DS (1979) *Computers and Intractability: A Guide to the Theory of NP-Completeness* (W. H. Freeman), ISBN 0-7167-1044-7.
- Gilmore PC, Gomory RE (1961) A linear programming approach to the cutting-stock problem. *Operations Research* 9(6):849–859.

- Gilmore PC, Gomory RE (1965) Multistage cutting stock problems of two and more dimensions. *Operations Research* 13(1):94–120.
- Gschwind T, Irnich S (2016) Dual inequalities for stabilized column generation revisited. *INFORMS Journal on Computing* 28(1):175–194.
- Guo S, Xiao F, Liang Z (2025) Nested set-covering/packing problem: Degeneracy alleviation and dual stabilization. *Operations Research* .
- Gzara F, Elhedhli S, Yildiz BC (2020) The pallet loading problem: Three-dimensional bin packing with practical constraints. *European Journal of Operational Research* 287(3):1062–1074.
- Held S, Cook W, Sewell EC (2012) Maximum-weight stable sets and safe lower bounds for graph coloring. *Mathematical Programming Computation* 4(4):363–381.
- Hong S, Zhang D, Lau HC, Zeng X, Si YW (2014) A hybrid heuristic algorithm for the 2D variable-sized bin packing problem. *European Journal of Operational Research* 238(1):95–103.
- IBM (2017) *IBM ILOG CPLEX Optimization Studio*. IBM, Armonk, NY, USA, <https://www.ibm.com/products/ilog-cplex-optimization-studio>.
- Iori M, De Lima VL, Martello S, Miyazawa FK, Monaci M (2021) Exact solution techniques for two-dimensional cutting and packing. *European Journal of Operational Research* 289(2):399–415.
- Iori M, de Lima VL, Martello S, Monaci M (2022) 2DPackLib: a two-dimensional cutting and packing library. *Optimization Letters* 16(2):471–480.
- Iori M, Salazar-González JJ, Vigo D (2007) An exact approach for the vehicle routing problem with two-dimensional loading constraints. *Transportation Science* 41(2):253–264.
- Jepsen M, Petersen B, Spoorendonk S, Pisinger D (2008) Subset-row inequalities applied to the vehicle-routing problem with time windows. *Operations Research* 56(2):497–511.
- Lai X, Hao JK, Yue D, Lü Z, Fu ZH (2022) Iterated dynamic thresholding search for packing equal circles into a circular container. *European Journal of Operational Research* 299(1):137–153.
- Lamas-Fernandez C, Bennell JA, Martinez-Sykora A (2023) Voxel-based solution approaches to the three-dimensional irregular packing problem. *Operations Research* 71(4):1298–1317.
- Liu C, Smith-Miles K, Wauters T, Costa AM (2024) Instance space analysis for 2D bin packing mathematical models. *European Journal of Operational Research* 315(2):484–498.
- Lodi A, Martello S, Monaci M, Cicconetti C, Lenzini L, Mingozzi E, Eklund C, Moilanen J (2011) Efficient two-dimensional packing algorithms for mobile WiMAX. *Management Science* 57(12):2130–2144.
- Lodi A, Martello S, Vigo D (1999) Heuristic and metaheuristic approaches for a class of two-dimensional bin packing problems. *INFORMS Journal on Computing* 11(4):345–357.
- Lodi A, Martello S, Vigo D (2002a) Heuristic algorithms for the three-dimensional bin packing problem. *European Journal of Operational Research* 141(2):410–420.

- Lodi A, Martello S, Vigo D (2002b) Recent advances on two-dimensional bin packing problems. *Discrete Applied Mathematics* 123(1-3):379–396.
- Lodi A, Monaci M, Pietrobuoni E (2017) Partial enumeration algorithms for two-dimensional bin packing problem with guillotine constraints. *Discrete Applied Mathematics* 217:40–47.
- Lübbecke ME, Desrosiers J (2005) Selected topics in column generation. *Operations Research* 53(6):1007–1023.
- Martello S (2014) Two-dimensional packing problems in telecommunications. *Pesquisa Operacional* 34:31–38.
- Martello S, Vigo D (1998) Exact solution of the two-dimensional finite bin packing problem. *Management Science* 44(3):388–399.
- Martin M, Yanasse HH, Santos MO, Morabito R (2024) Models for two-dimensional bin packing problems with customer order spread. *Journal of Combinatorial Optimization* 48(1):8.
- Monaci M, Toth P (2006) A set-covering-based heuristic approach for bin-packing problems. *INFORMS Journal on Computing* 18(1):71–85.
- Parreño F, Alvarez-Valdés R, Oliveira JF, Tamarit JM (2010) A hybrid GRASP/VND algorithm for two-and three-dimensional bin packing. *Annals of Operations Research* 179(1):203–220.
- Pecin D, Pessoa A, Poggi M, Uchoa E (2017) Improved branch-cut-and-price for capacitated vehicle routing. *Mathematical Programming Computation* 9(1):61–100.
- Pessoa A, Sadykov R, Uchoa E, Vanderbeck F (2020) A generic exact solver for vehicle routing and related problems. *Mathematical Programming* 183(1):483–523.
- Pisinger D, Sigurd M (2007) Using decomposition techniques and constraint programming for solving the two-dimensional bin-packing problem. *INFORMS Journal on Computing* 19(1):36–51.
- Ryan DM, Foster BA (1981) An integer programming approach to scheduling. *Computer Scheduling of Public Transport Urban Passenger Vehicle and Crew Scheduling* 269–280.
- Sadykov R, Vanderbeck F, Pessoa A, Tahiri I, Uchoa E (2019) Primal heuristics for branch and price: The assets of diving methods. *INFORMS Journal on Computing* 31(2):251–267.
- Salih NK, Gopal R (2024) Optimization software fault injection by using two-dimensional bin packing algorithms. *Journal of Electrical Systems* 20(10s).
- Serairi M, Haouari M (2018) A theoretical and experimental study of fast lower bounds for the two-dimensional bin packing problem. *RAIRO-Operations Research-Recherche Opérationnelle* 52(2):391–414.
- Sun B, Li G, Wang S, Xie C (2025) Two-dimensional bin-packing problem with conflicts and load balancing: A hybrid chaotic and evolutionary particle swarm optimization algorithm. *Computers & Industrial Engineering* 200:110851.
- Turbian C, DE GIOVANNI L, Gastaldon N, et al. (2022) A two-dimensional bin packing problem with precedence constraints in the sheet metal industry. *ODS2022–Book of abstracts*, 110–110 (ODS Publishing).

- Uchoa E, Pessoa A, Moreno L (2024) Optimizing with Column Generation: Advanced branch-cut-and-price algorithms (Part I). Technical Report L-2024-3, Cadernos do LOGIS-UFF, Universidade Federal Fluminense, Engenharia de Produção, URL <https://optimizingwithcolumngeneration.github.io>.
- Valério de Carvalho J (1999) Exact solution of bin-packing problems using column generation and branch-and-bound. *Annals of Operations Research* 86(0):629–659.
- Wang S, Baldacci R, Furini F, Wei L, Liu Q (2025a) Exact algorithms for two-dimensional knapsack problems: A unified framework with new benchmark results. *Available at SSRN 5322387*.
- Wang S, Baldacci R, Liu Q, Wei L (2025b) EATKG: An open-source efficient exact algorithm for the two-dimensional knapsack problem with guillotine constraints. *European Journal of Operational Research*.
- Wang S, Wang R, Zhang H, Jing F, Liu Q, Wei L (2025c) A goal-driven iterated local search approach based on the maximal-space for the circle bin-packing problem with rectangular items, manuscript currently under review at *Computers & Operations Research* (two major revisions completed).
- Wang S, Yao S, Zhang H, Liu Q, Wei L (2025d) Arc-flow formulation and branch-and-price-and-cut algorithm for the bin-packing problem with fragile objects. *Computers & Operations Research* 173:106878.
- Wang Z, Chang D, Man X (2022) Optimization of two-dimensional irregular bin packing problem considering slit distance and free rotation of pieces. *International Journal of Industrial Engineering Computations* 13(4):491–506.
- Wei L, Lai M, Lim A, Hu Q (2020a) A branch-and-price algorithm for the two-dimensional vector packing problem. *European Journal of Operational Research* 281(1):25–35.
- Wei L, Luo Z, Baldacci R, Lim A (2020b) A new branch-and-price-and-cut algorithm for one-dimensional bin-packing problems. *INFORMS Journal on Computing* 32(2):428–443.
- Wei L, Oon WC, Zhu W, Lim A (2013) A goal-driven approach to the 2D bin packing and variable-sized bin packing problems. *European Journal of Operational Research* 224(1):110–121.
- Xu Z, Lee CY (2018) New lower bound and exact method for the continuous berth allocation problem. *Operations Research* 66(3):778–798.
- Zhang H, Liu Q, Wei L, Zeng J, Leng J, Yan D (2022a) An iteratively doubling local search for the two-dimensional irregular bin packing problem with limited rotations. *Computers & Operations Research* 137:105550.
- Zhang X, Chen L, Gendreau M, Langevin A (2022b) Learning-based branch-and-price algorithms for the vehicle routing problem with time windows and two-dimensional loading constraints. *INFORMS Journal on Computing* 34(3):1419–1436.
- Zhang Z, Guo S, Zhu W, Oon WC, Lim A (2011) Space defragmentation heuristic for 2D and 3D bin packing problems. *IJCAI Proceedings-International Joint Conference on Artificial Intelligence*, volume 22, 699.
- Zhou J, He J, He K (2025) Solution-hashing search based on layout-graph transformation for unequal circle packing. *European Journal of Operational Research*.

Zhu W, Chen S, Dai M, Tao J (2024) Solving a 3D bin packing problem with stacking constraints. *Computers & Industrial Engineering* 188:109814.

E-Companion to the Paper Titled “A Highly Efficient Numerically Exact Algorithm for Two-Dimensional Bin-Packing Problems”

EC.1. Additional Algorithmic Details

EC.1.1. Computing the Initial Lower Bound for 2BPP(-G)

The initial lower bound LB_0 is computed as the maximum of three polynomial 2BPP lower bounds from the literature: $LB_{BM,3}^2$, $LB_{CCM,1}^2$, and $LB_{CCM,2}^2$, i.e., $LB_0 = \max \{LB_{BM,3}^2, LB_{CCM,1}^2, LB_{CCM,2}^2\}$. Note that since the 2BPP is a relaxation of the 2BPP-G, these 2BPP lower bounds are also valid for 2BPP-G.

The lower bound $LB_{BM,3}^2$ proposed by [Boschetti and Mingozi \(2003\)](#) considers both dimensions simultaneously. Given two integers p and q such that $1 \leq p \leq H/2$ and $1 \leq q \leq W/2$, define the following item subsets:

$$\mathcal{I}_{\text{Large}}(p, q) = \{i \in \mathcal{I} : w_i > W - q \text{ and } h_i > H - p\}, \quad (\text{EC.1a})$$

$$\mathcal{I}_{\text{Medium}}(p, q) = \{i \in \mathcal{I} \setminus \mathcal{I}_{\text{Large}}(p, q) : w_i > W/2 \text{ and } h_i > H/2\}. \quad (\text{EC.1b})$$

Since any two items belonging to $\mathcal{I}_{\text{Large}}(p, q) \cup \mathcal{I}_{\text{Medium}}(p, q)$ cannot be packed together in the same bin, $|\mathcal{I}_{\text{Large}}(p, q) \cup \mathcal{I}_{\text{Medium}}(p, q)|$ provides a valid lower bound. Furthermore, we consider a subset $\mathcal{I}_s(p, q)$ of the remaining items and compute a lower bound on the number of bins required to pack the items of this subset, which is defined as follows:

$$\mathcal{I}_s(p, q) = \{i \in \mathcal{I} \setminus (\mathcal{I}_{\text{Large}}(p, q) \cup \mathcal{I}_{\text{Medium}}(p, q)) : w_i \geq q \text{ and } h_i \geq p\}. \quad (\text{EC.2})$$

Let $M^W(w, \mathcal{I}_s(p, q))$ denote the maximum number of items from $\mathcal{I}_s(p, q)$ that can be packed side by side in a bin of width w . By symmetry, $M^H(h, \mathcal{I}_s(p, q))$ is defined similarly, considering the height dimension. An upper bound on the number of items from $\mathcal{I}_s(p, q)$ that can be packed into an empty bin is given by $M^W(w, \mathcal{I}_s(p, q)) \times M^H(h, \mathcal{I}_s(p, q))$. Additionally, an upper bound on the number of items that can be packed into a bin that has been initialized with an item i from $\mathcal{I}_{\text{Medium}}(p, q)$ is given by:

$$\begin{aligned} m(i, p, q) = & M^W(W - w_i, \mathcal{I}_s(p, q)) \times M^H(H, \mathcal{I}_s(p, q)) + M^W(W, \mathcal{I}_s(p, q)) \times M^H(H - h_i, \mathcal{I}_s(p, q)) \\ & - M^W(W - w_i, \mathcal{I}_s(p, q)) \times M^H(H - h_i, \mathcal{I}_s(p, q)). \end{aligned} \quad (\text{EC.3})$$

Finally, the lower bound $LB_{BM,3}^2$ is calculated as:

$$LB_{BM,3}^2 = \max_{\substack{1 \leq p \leq H/2 \\ 1 \leq q \leq W/2}} \left\{ |\mathcal{I}_{\text{Large}}(p, q) \cup \mathcal{I}_{\text{Medium}}(p, q)| + \max \left\{ 0, \left\lceil \frac{|\mathcal{I}_s(p, q)| - \sum_{i \in \mathcal{I}_{\text{Medium}}(p, q)} m(i, p, q)}{M^W(W, \mathcal{I}_s(p, q)) \times M^H(H, \mathcal{I}_s(p, q))} \right\rceil \right\} \right\}, \quad (\text{EC.4})$$

The lower bounds $LB_{CCM,1}^2$ and $LB_{CCM,2}^2$ are based on the concept of the dual feasible function (DFF) and the discrete DFF, which are the foundations of the most recent and effective lower bounding techniques for the 2BPP. Before proceeding further, we present two basic definitions (see Definitions 1 and 2) of these functions.

Definition 1 A function $f : [0, 1] \rightarrow [0, 1]$ is a DFF if it satisfies the following condition for any finite set $\mathcal{S}$ of positive real numbers: $\sum_{x \in \mathcal{S}} x \leq 1 \implies \sum_{x \in \mathcal{S}} f(x) \leq 1$.

Definition 2 A function $f : [0, C] \rightarrow [0, C']$ is a discrete DFF if it satisfies the following condition for any finite set $\mathcal{S}$ of integer numbers: $\sum_{x \in \mathcal{S}} x \leq C \implies \sum_{x \in \mathcal{S}} f(x) \leq f(C) = C'$.

The concept of DFFs was first introduced by Johnson (1973) in the context of bin-packing. Furthermore, Carlier and Néron (2007) introduced the concept of *redundant functions* in the context of cumulative scheduling, which can be viewed as discretized versions of DFFs. Several DFFs have since been used to solve the bin-packing problem. We recommend that readers interested in DFFs refer to the survey by Clautiaux et al. (2010). In the following, we provide only a brief review of what is necessary to calculate $LB_{\text{CCM},1}^2$ and $LB_{\text{CCM},2}^2$.

The first discrete DFF (EC.5) proposed by Carlier et al. (2007) is used to modify the width or height of items and bins. The new area is calculated by trying one or more combinations of the modified width and height, which is then used to calculate the new area bound.

$$f_1(k, x, C, \cdot) = \begin{cases} C, & \text{if } C - k < x, \\ x, & \text{if } k \leq x \leq C - k, \\ 0, & \text{otherwise.} \end{cases} \quad (\text{EC.5})$$

The second discrete DFF (EC.6) proposed by Carlier et al. (2007) is a discrete data-dependent DFF. Let $\text{CKP}(C, \tilde{\mathcal{I}})$ be a *cardinality knapsack problem* (CKP), where the objective is to maximize the number of items in $\tilde{\mathcal{I}}$ that can fit into a bin of capacity C , with $z(\text{CKP}(C, \tilde{\mathcal{I}}))$ denoting its optimal solution value.

$$f_2(k, x, C, \tilde{\mathcal{I}}) = \begin{cases} z(\text{CKP}(C, \tilde{\mathcal{I}})) - z(\text{CKP}(C - x, \tilde{\mathcal{I}})), & \text{if } C/2 < x, \\ 1, & \text{if } k \leq x \leq C/2, \\ 0, & \text{otherwise.} \end{cases} \quad (\text{EC.6})$$

The third discrete DFF, given by (EC.7), was also proposed by Carlier et al. (2007) and is a maximal DFF.

$$f_3(k, x, C, I) = \begin{cases} 2(\lfloor C/k \rfloor - \lfloor (C - x)/k \rfloor), & \text{if } C/2 < x, \\ \lfloor C/k \rfloor, & \text{if } x = C/2, \\ 2\lfloor x/k \rfloor, & \text{if } x < C/2. \end{cases} \quad (\text{EC.7})$$

We begin with $LB_{\text{CCM},1}^2$, which can be viewed as a modified version of $LB_{\text{BM},2}^2$. Given two integers p and q such that $1 \leq p \leq H/2$ and $1 \leq q \leq W/2$, we define the following three subsets of items:

$$\mathcal{I}_{\text{Tall}}(p, q) = \{i \in \mathcal{I} \setminus \mathcal{I}_{\text{Large}}(p, q) : w_i \geq q \text{ and } h_i > H - p\}, \quad (\text{EC.8a})$$

$$\mathcal{I}_{\text{Wide}}(p, q) = \{i \in \mathcal{I} \setminus \mathcal{I}_{\text{Large}}(p, q) : w_i > W - q \text{ and } h_i \geq p\}, \quad (\text{EC.8b})$$

$$\mathcal{I}_{\text{Small}}(p, q) = \{i \in \mathcal{I} \setminus (\mathcal{I}_{\text{Large}}(p, q) \cup \mathcal{I}_{\text{Tall}}(p, q) \cup \mathcal{I}_{\text{Wide}}(p, q)) : w_i \geq q \text{ and } h_i \geq p\}. \quad (\text{EC.8c})$$

Since each item in $\mathcal{I}_{\text{Large}}(p, q)$ requires a separate bin, $|\mathcal{I}_{\text{Large}}(p, q)|$ provides a valid lower bound. Moreover, there is no item from $\mathcal{I}_{\text{Tall}}(p, q) \cup \mathcal{I}_{\text{Wide}}(p, q) \cup \mathcal{I}_{\text{Small}}(p, q)$ that can be packed together with an item from $\mathcal{I}_{\text{Large}}(p, q)$. Consequently, $|\mathcal{I}_{\text{Large}}(p, q)|$ can be tightened by computing a lower bound on the number of bins required to pack the items of $\mathcal{I}_{\text{Tall}}(p, q) \cup \mathcal{I}_{\text{Wide}}(p, q) \cup \mathcal{I}_{\text{Small}}(p, q)$. $LB_{\text{CCM},1}^2$ can be calculated by converting

a 2BPP instance composed of the items from $\mathcal{I}_{\text{Tall}}(p, q) \cup \mathcal{I}_{\text{Wide}}(p, q) \cup \mathcal{I}_{\text{Small}}(p, q)$ into a BPP instance I^{BPP} .

Consider the following three BPP instances:

- $I_1^{\text{BPP}}(p, q) : \mathcal{I}^{\text{BPP}} = \mathcal{I}_{\text{Tall}}(p, q) \cup \mathcal{I}_{\text{Wide}}(p, q) \cup \mathcal{I}_{\text{Small}}(p, q), \quad C = WH, \text{ and } c_i = \begin{cases} w_i H & \text{if } i \in \mathcal{I}_{\text{Tall}}(p, q) \\ h_i W & \text{if } i \in \mathcal{I}_{\text{Wide}}(p, q) \\ w_i h_i & \text{if } i \in \mathcal{I}_{\text{Small}}(p, q) \end{cases}$
- $I_2^{\text{BPP}}(p, q) : \mathcal{I}^{\text{BPP}} = \mathcal{I}_{\text{Wide}}(p, q), \quad C = H, \text{ and } c_i = h_i.$
- $I_3^{\text{BPP}}(p, q) : \mathcal{I}^{\text{BPP}} = \mathcal{I}_{\text{Tall}}(p, q), \quad C = W, \text{ and } c_i = w_i.$

Consequently, a second valid lower bound on the minimal number of bins required to pack the items belonging to $\mathcal{I}_{\text{Tall}}(p, q) \cup \mathcal{I}_{\text{Wide}}(p, q) \cup \mathcal{I}_{\text{Small}}(p, q)$ is $LB_{\text{CCM}}^1(I_2^{\text{BPP}}(p, q)) + LB_{\text{CCM}}^1(I_3^{\text{BPP}}(p, q))$, where $LB_{\text{CCM}}^1(I^{\text{BPP}}) = \max_{u, k} \sum_{i \in \mathcal{I}^{\text{BPP}}} \frac{f_u(k, c_i, C, \mathcal{I}^{\text{BPP}})}{f_u(k, C, C, \mathcal{I}^{\text{BPP}})}$. Then, the lower bound $LB_{\text{CCM},1}^2$ is calculated as

$$LB_{\text{CCM},1}^2 = \max_{p, q} |\mathcal{I}_{\text{Large}}(p, q)| + \max \{LB_{\text{CCM}}^1(I_1^{\text{BPP}}(p, q)), LB_{\text{CCM}}^1(I_2^{\text{BPP}}(p, q)) + LB_{\text{CCM}}^1(I_3^{\text{BPP}}(p, q))\}. \quad (\text{EC.9})$$

Considering the trade-off between computation time and lower bound quality, set $p \in \{1, 1 + \frac{H}{40}, 1 + \frac{2H}{40}, \dots, H/2\}$, $q \in \{1, 1 + \frac{W}{40}, 1 + \frac{2W}{40}, \dots, W/2\}$, $u \in \{1, 2, 3\}$, and $k \in \{1, 1 + \frac{C}{40}, 1 + \frac{2C}{40}, \dots, C/2\}$.

Finally, we introduce the lower bound $LB_{\text{CCM},2}^2$. Three discrete DFFs are calculated for all possible parameter values and are combined in all possible ways to obtain the highest lower bound:

$$LB_{\text{CCM},2}^2 = \max_{u, v \in \{1, 2, 3\}, k \in \{1, 2, \dots, W/2\}, l \in \{1, 2, \dots, H/2\}} \left[\sum_{i \in \mathcal{I}} \frac{f_u(k, w_i, W, \mathcal{I}_w^k) f_v(l, h_i, H, \mathcal{I}_h^l)}{f_u(k, W, W, \mathcal{I}_w^k) f_v(l, H, H, \mathcal{I}_h^l)} \right], \quad (\text{EC.10})$$

where $\mathcal{I}_w^k = \{i \in \mathcal{I} : W/2 \leq w_i \leq k\}$ and $\mathcal{I}_h^l = \{i \in \mathcal{I} : H/2 \leq h_i \leq l\}$.

EC.1.2. Triple-Block-Based Heuristic

The following discussion assumes familiarity with the steps of the heuristic algorithm outlined in Section 3 of Cui et al. (2018). We adapt their heuristic method by additionally applying a post-improvement procedure after Step 7. In the procedure, we apply a tabu search packing algorithm, see Algorithm 1 in Wang et al. (2025a), to each pair of adjacent bins in the current solution, attempting to pack more items into the former. If all items are packed into the former, the latter is removed from the current solution, and a better solution is obtained to update the initial upper bound. Otherwise, by traversing all pairs of bins in the current solution, the tabu search is used to attempt to pack all items in the two bins into one bin, thereby obtaining an improved solution. Note that when solving 2BPP-G, we replace the core packing algorithm (a skyline-based heuristic) of the tabu search with the BFH1 from Wang et al. (2025b).

EC.1.3. Goal-Driven Iterated Local Search Heuristic

Algorithm 1 outlines the main steps of the goal-driven heuristic, which are briefly described as follows. The heuristic (Lines 1-4) iteratively applies an iterated local search procedure (see §EC.1.3.1) to search for a 2BPP(-G) solution (sol) with a cost less than or equal to $UB_0 - 1$ (Line 2). If all items in $\mathcal{I}$ are packed in sol , the incumbent solution $\widetilde{sol}$ and the initial upper bound UB_0 are updated (Line 4). The iteration continues until the time limit is reached or $LB_0 = UB_0$.

EC.1.3.1. Iterated Local Search. Algorithm 2 outlines the iterated local search procedure, which incorporates a tabu search packing subroutine (see §EC.1.3.2) for local optimization, a squeeze subroutine to attempt improving the current solution, and a perturbation operation to escape local optima. In Line 1, three parameters, β , ζ , and η , used in tabu search are initialized as β_0 , ζ_0 , and η_0 (their various values are analyzed in §EC.2.1). The parameter η is related to the number of iterations of the tabu search; it is iteratively updated in Line 7 to gradually increase the computational effort of tabu search with an upper limit. If $target \leq 3$, tabu search usually takes less time, so we gradually increase the η value; otherwise, we increase the η value more quickly to ensure that it reaches the upper limit within the time limit.

The squeeze (SQUEEZE) subroutine, adapted from the method of Parreño et al. (2010), takes the current solution sol as input and is executed in a local search manner. The move involves emptying the two bins in sol and completing the resulting partial solution. The first item placed in the first bin must have the largest area among the unpacked items. The neighborhood consists of all pairs of bins, where the total area of items in two bins plus the area of unpacked items must not exceed $2WH$. This subroutine allows for the packing of previously challenging items and potentially improves the current solution. Figure EC.1 shows an example of the squeeze subroutine, where bins 2 and 4 are emptied (highlighted with red borders). The only item (shown with a gray fill pattern) not included in any of the bins in the solution is packed in the first empty bin, and the remaining items are packed using the skyline-based heuristic from Wang et al. (2025a) when solving 2BPP or using BFH1 from Wang et al. (2025b) when solving 2BPP-G.

The perturbation operation is applied to the current solution sol and its corresponding item sequence $\mathcal{I}_s$. Let $\mathcal{B}_{sol}$ be the set of bins used by sol . Initially, a integer k is uniformly and randomly selected from the range $[1, |\mathcal{B}_{sol}|]$. Then, k bins are randomly emptied from sol . Next, the skyline-based heuristic or BFH1 is applied to pack the unpacked items into the emptied bins. If any items remain unpacked, for each bin $b \in \mathcal{B}_{sol}$, we empty bin b and then employ the skyline-based heuristic or BFH1 to attempt packing the unpacked items into bin b . If any items remain unpacked, we reorder the item sequence $\mathcal{I}_s$ based on the current solution sol and the following sorting rules: start with the item with the smallest bin index; ties are broken in favor of items with the smallest y -coordinate, and second-level ties are broken in favor of items

Algorithm 1: GOALDRIVENHEURISTIC($\widetilde{sol}, \widetilde{\mathcal{I}}$)

Input : Incumbent solution $\widetilde{sol}$ and item sequence $\widetilde{\mathcal{I}}$ associated with the best solution obtained by the fit-based heuristic

Output: Updated solution $\widetilde{sol}$ and initial upper bound UB_0

```

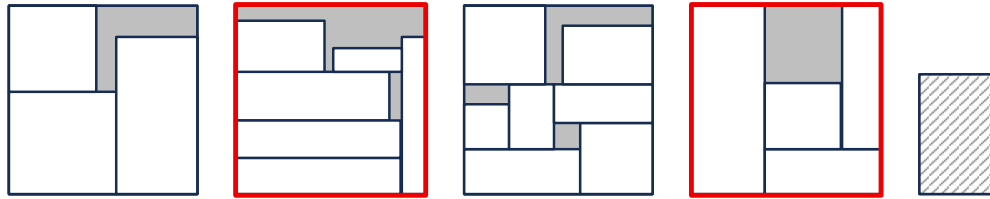
1 while time limit is not reached and  $LB_0 < UB_0$  do
2    $sol, \mathcal{I}_s \leftarrow \text{ITERATEDLOCALSEARCH}(UB_0 - 1, \widetilde{sol}, \widetilde{\mathcal{I}})$  (see Algorithm 2 in §EC.1.3.1)
3   if all items are packed in  $sol$  then
4      $\widetilde{sol} \leftarrow sol; \widetilde{\mathcal{I}} \leftarrow \mathcal{I}_s; UB_0 \leftarrow$  the number of bins in  $sol$ 
```

Algorithm 2: ITERATEDLOCALSEARCH($target, sol, \mathcal{I}_s$)**Input** : Target number of bins ($target$), incumbent solution (sol), and item sequence ($\mathcal{I}_s$)**Output**: Best solution found sol (or sol') and item sequence $\mathcal{I}_s$

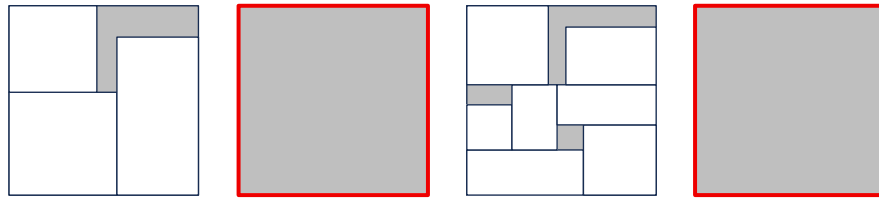
```

1  $\beta \leftarrow \beta_0; \zeta \leftarrow \zeta_0; \eta \leftarrow \eta_0$ 
2 while time limit is not reached do
3    $sol, \mathcal{I}_s \leftarrow \text{TABUSEARCHPACKTARGET}(\beta, \zeta, \eta, target, sol, \mathcal{I}_s)$  (see Algorithm 3 in §EC.1.3.2)
4   if all items are packed in  $sol$  then return  $sol, \mathcal{I}_s$ 
5   if  $LB_0 = target$  then  $\{sol' \leftarrow \text{SQUEEZE}(sol); \text{if all items are packed in } sol' \text{ then return } sol', \mathcal{I}_s\}$ 
6    $sol, \mathcal{I}_s \leftarrow \text{PERTURBATION}(sol, \mathcal{I}_s); \text{if all items are packed in } sol \text{ then return } sol, \mathcal{I}_s$ 
7   if  $target \leq 3$  then  $\eta \leftarrow \min\{6\eta_0, \eta + \eta_0\}$  else  $\eta \leftarrow \min\{8\eta_0, \eta + 4\eta_0\}$ 
8 return  $sol, \mathcal{I}_s$ 

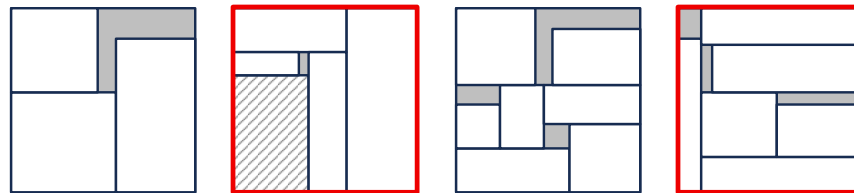
```



(a) Current (incomplete) solution with an unpacked item



(b) Empty two bins



(c) Pack the unpacked items

Figure EC.1 Illustration of an Execution Example of the Squeeze Subroutine

with the smallest x -coordinate. Note that the rules only apply to items that have been packed. Unpacked items are directly appended to the end of the item sequence in their original order.

Algorithm 3: TABUSEARCHPACKTARGET($\beta, \zeta, \eta, target, sol_0, \mathcal{I}_0$)**Input** : Parameters (β, ζ, η), target number of bins ($target$), initial solution (sol_0), and item sequence ($\mathcal{I}_0$)**Output:** Best solution found sol^* (or sol) and its corresponding sequence of items $\mathcal{I}^*$ (or $\mathcal{I}$)

```

1 Initialize a tabu list to be empty;  $sol^* \leftarrow sol_0$ ;  $\mathcal{I}^* \leftarrow \mathcal{I}_0$ ;  $noImp \leftarrow 0$ 
2 while time limit is not reached and  $noImp < \eta|\mathcal{I}_0|$  do
3   Generate  $\beta$  non-tabu sequences  $\{\mathcal{I}'_1, \mathcal{I}'_2, \dots, \mathcal{I}'_\beta\}$  from  $\mathcal{I}^*$  by swapping two items;  $improve \leftarrow \text{FALSE}$ 
4   foreach  $\mathcal{I}' \in \{\mathcal{I}'_1, \mathcal{I}'_2, \dots, \mathcal{I}'_\beta\}$  do
5      $sol \leftarrow \text{FITBASEDPACKTARGET}(target, \mathcal{I}')$ ; if all items are packed in  $sol$  then return  $sol, \mathcal{I}'$ 
6     if  $fitness(sol) > fitness(sol^*)$  then  $\{sol^* \leftarrow sol, \mathcal{I}^* \leftarrow \mathcal{I}'; improve \leftarrow \text{TRUE}\}$ 
7   Let  $\{i, j\}$  be the items swapped to produce the best sequence in  $\{\mathcal{I}'_1, \mathcal{I}'_2, \dots, \mathcal{I}'_\beta\}$ 
8   Add  $\{i, j\}$  to the tabu list for the next  $\zeta|\mathcal{I}_0|$  iterations
9   if  $improve$  is TRUE then  $noImp \leftarrow 0$  else  $noImp \leftarrow noImp + 1$ 
10 return  $sol^*, \mathcal{I}^*$ 

```

EC.1.3.2. Tabu Search. The tabu search packing subroutine is outlined in Algorithm 3. Initially, the tabu list is set as an empty set. In each iteration (Lines 2-9), we randomly select two items, i and j , in $\mathcal{I}^*$. If $\{i, j\}$ is not in the tabu list, we swap them to produce a new sequence of items. This way, we generate up to β non-tabu sequences (Line 3). From these β non-tabu sequences, we select the one with the largest fitness value (as explained below) and insert the swap into the tabu list; this swap is forbidden for the next $\zeta|\mathcal{I}_0|$ iterations (Lines 4-8). We use a positive integer $noImp$ to record the number of times the current best solution has not been improved (Line 9). Once $noImp \geq \eta|\mathcal{I}_0|$ or the time limit is reached, the loop breaks and returns sol^* and $\mathcal{I}^*$ (Line 10).

To evaluate the fitness value of an item sequence, we first apply the fit-based heuristic to the sequence to obtain an (incomplete) solution sol with at most $target$ bins used. Then, let $\mathcal{I}_b$ be the set of items packed into bin $b \in \mathcal{B}_{sol}$. The fitness function used is defined as $fitness(sol) = \sum_{b \in \mathcal{B}_{sol}} \left(\sum_{i \in \mathcal{I}_b} w_i h_i \right)^2$. This fitness function, by awarding higher scores to solutions with unbalanced packings, promotes the selection of such solutions, thereby enhancing the probability of emptying bins with smaller packed item areas in the next iteration.

EC.1.4. Rounding Heuristic

The rounding heuristic is based on a rounding subroutine, which takes a parameter ϵ as input and is outlined in Algorithm 4. Without loss of generality, we assume that the patterns p in $\overline{\mathcal{P}}$ are sorted in non-increasing ξ_p . Initially, based on the parameter ϵ , we construct a subset $\overline{\mathcal{P}}'$ of $\overline{\mathcal{P}}$ consisting of the first $UB - 1$ patterns from $\{p \in \overline{\mathcal{P}} : \xi_p \geq 1 - \epsilon\}$ (Line 1). In the for-loop (Lines 2-5), we attempt to pack items that have not yet been included in $\overline{\mathcal{P}}'$ into a pattern of $\overline{\mathcal{P}}'$, where p^i is the i -th pattern in $\overline{\mathcal{P}}'$. Then, let $\mathcal{I}'$ be the set of items not packed in any pattern $p \in \overline{\mathcal{P}}'$ (Line 6). If there are still unpacked items (i.e., $\mathcal{I}' \neq \emptyset$, Line 7), another for-loop (Lines 8-14) is used to attempt packing the unpacked items in $\mathcal{I}'$ into the patterns in $\overline{\mathcal{P}}'$.

Algorithm 4: ROUNDINGSUBROUTINE(ϵ)**Input** : Parameter ϵ **Output:** Updated incumbent solution value UB

```

1  $\overline{\mathcal{P}}' \leftarrow$  first  $UB - 1$  patterns of set  $\{p \in \overline{\mathcal{P}} : \xi_p \geq 1 - \epsilon\}$ 
2 foreach  $i \in \{1, 2, \dots, |\overline{\mathcal{P}}'|\}$  do
3    $\mathcal{I}_{\text{front}}^i \leftarrow \bigcup_{j \in \{1, 2, \dots, i-1\}} \mathcal{I}_{p^j}; \mathcal{I}_{\text{back}}^i \leftarrow \bigcup_{j \in \{i+1, i+2, \dots, |\overline{\mathcal{P}}'|\}} \mathcal{I}_{p^j}; \mathcal{I}^i \leftarrow \mathcal{I} \setminus (\mathcal{I}_{\text{front}}^i \cup \mathcal{I}_{\text{back}}^i)$ 
4    $p^* \leftarrow \text{TABUSEARCHPACK}(W, H, \mathcal{I}^i); \mathcal{I}_{p^i} \leftarrow \mathcal{I}^i$ 
5   if all items in  $\mathcal{I}^i$  are packed in  $p^*$  then break
6  $\mathcal{I}' \leftarrow \mathcal{I} \setminus \bigcup_{p \in \overline{\mathcal{P}}'} \mathcal{I}_p$ 
7 if  $\mathcal{I}' \neq \emptyset$  then
8   foreach  $i \in \{1, 2, \dots, |\overline{\mathcal{P}}'|\}$  do
9     if  $A(\mathcal{I}_{p^i}) + A(\mathcal{I}') \leq WH$  then
10      if all items in  $\mathcal{I}_{p^i} \cup \mathcal{I}'$  can be packed in a bin (i.e., they pass the exact 2OPP(-G) check) then
11         $\mathcal{I}_{p^i} \leftarrow \mathcal{I}_{p^i} \cup \mathcal{I}'; \text{break}$ 
12      else
13         $p^* \leftarrow \text{TABUSEARCHPACK}(W, H, \mathcal{I}_{p^i} \cup \mathcal{I}')$ 
14        if all items in  $\mathcal{I}_{p^i} \cup \mathcal{I}'$  are packed in  $p^*$  then  $\{\mathcal{I}_{p^i} \leftarrow \mathcal{I}_{p^i} \cup \mathcal{I}'; \text{break}\}$ 
15 if  $\mathcal{I} \setminus \bigcup_{p \in \overline{\mathcal{P}}'} \mathcal{I}_p = \emptyset$  then  $UB \leftarrow |\overline{\mathcal{P}}'|$ 

```

Note that an exact check for the two-dimensional orthogonal packing problem (with guillotine constraints) (denoted as 2OPP(-G)) is required multiple times within this for-loop. The exact BB algorithm introduced in §3.6.2 of Wang et al. (2025a) is used to perform the 2OPP check. For the 2OPP(-G) check, we adapt the BB algorithm by removing inner BB nodes whose corresponding patterns violate the guillotine constraints. Furthermore, to avoid excessive time consumption, we limit each exact 2OPP(-G) check to a maximum of 0.5 seconds. Finally, if $\mathcal{I} \setminus \bigcup_{p \in \overline{\mathcal{P}}'} \mathcal{I}_p = \emptyset$, it indicates that the rounding subroutine has found a solution with the number of bins used being $UB - 1$ (Line 15), which is better than the incumbent solution. Note that “TABUSEARCHPACK” in Algorithm 4 refers to Algorithm 1 in the Appendix of Wang et al. (2025a). When solving 2BPP-G, we replace the core packing algorithm in “TABUSEARCHPACK” with the BFH1 from Wang et al. (2025b).

Since different values of the parameter ϵ may lead to different $\overline{\mathcal{P}}'$ (Line 1), thereby affecting the rounding results, the proposed rounding heuristic executes the rounding subroutine multiple times with different values of ϵ . Specifically, ϵ is set to 0.4 at the root node. When the depth reaches 1, ϵ takes values of 0.4, 0.6, and 0.8. At nodes with a depth of 2, ϵ varies between 0.2 and 0.8 in increments of 0.2. For all other nodes at greater depths, ϵ ranges from 0.1 to 0.9 in increments of 0.1. This strategy is designed to respond to increasing depth by incrementing the frequency of calls to the subroutine and enhancing the likelihood of finding an improved primal solution.

Table EC.1 Results of the Parameter Calibration of the Hybrid Heuristic

β_0	ζ_0	η_0	#UB ₀	Time	β_0	ζ_0	η_0	#UB ₀	Time	β_0	ζ_0	η_0	#UB ₀	Time	β_0	ζ_0	η_0	#UB ₀	Time
5	1	1	7231	2.013	10	1	1	7230	1.741	15	1	1	7229	1.710	20	1	1	7229	1.709
5	1	3	7230	1.764	10	1	3	7230	1.742	15	1	3	7229	1.710	20	1	3	7229	1.709
5	1	5	7230	1.741	10	1	5	7229	1.750	15	1	5	7229	1.709	20	1	5	7229	1.709
5	1	7	7230	1.747	10	1	7	7229	1.710	15	1	7	7229	1.709	20	1	7	7229	1.710
5	3	1	7230	1.745	10	3	1	7229	1.710	15	3	1	7229	1.710	20	3	1	7229	1.709
5	3	3	7230	1.755	10	3	3	7229	1.711	15	3	3	7229	1.710	20	3	3	7229	1.708
5	3	5	7230	1.746	10	3	5	7229	1.711	15	3	5	7229	1.710	20	3	5	7229	1.710
5	3	7	7230	1.746	10	3	7	7229	1.711	15	3	7	7229	1.714	20	3	7	7229	1.709
5	5	1	7230	1.751	10	5	1	7229	1.717	15	5	1	7229	1.709	20	5	1	7229	1.709
5	5	3	7230	1.748	10	5	3	7229	1.710	15	5	3	7229	1.709	20	5	3	7229	1.708
5	5	5	7230	1.738	10	5	5	7229	1.710	15	5	5	7229	1.709	20	5	5	7229	1.710
5	5	7	7230	1.738	10	5	7	7229	1.710	15	5	7	7229	1.709	20	5	7	7229	1.709
5	7	1	7230	1.738	10	7	1	7229	1.706	15	7	1	7229	1.709	20	7	1	7229	1.710
5	7	3	7230	1.752	10	7	3	7229	1.709	15	7	3	7229	1.709	20	7	3	7229	1.708
5	7	5	7230	1.742	10	7	5	7229	1.710	15	7	5	7229	1.709	20	7	5	7229	1.709
5	7	7	7230	1.742	10	7	7	7229	1.709	15	7	7	7229	1.709	20	7	7	7229	1.710

EC.2. Additional Computational Experiments and Analyses

In this section, we analyze the proposed algorithm and its main components on 500 classic benchmark instances from [Berkey and Wang \(1987\)](#) and [Martello and Vigo \(1998\)](#), with a one-hour time limit for NE-BPC. Note that in the following tables, the meanings of the columns are the same as in the main paper, except for the “Time” column, which represents the average computation time without scaling based on the CPU score.

EC.2.1. Parameter Calibration for the Hybrid Heuristic

Recall that the proposed hybrid heuristic sequentially applies the fit-based heuristic, the triple-block-based heuristic, and the goal-driven heuristic. The fit-based heuristic does not involve any parameters, and its execution is computationally inexpensive; therefore, no time limit is imposed. The algorithm parameters of the triple-block-based heuristic follow the settings in [Cui et al. \(2018\)](#), with the only difference being that we impose a 6-second time limit on the first phase (compared to their 5 seconds). This adjustment is based on our preliminary experiments, which showed that increasing the time limit by 1 second results in better results without significantly increasing the average computation time. The goal-driven heuristic has an adaptive time limit of $\frac{n}{8}$ seconds. Additionally, its parameters β_0 , ζ_0 , and η_0 are set to 10, 7, and 1, respectively, based on the calibration results in [Table EC.1](#).

EC.2.2. Analysis of the Impact of the Scaling Factor

[Table EC.2](#) analyzes the impact of varying the scaling factor $K \in \{10^9, 10^{10}, 10^{11}, 10^{12}, 10^{13}\}$. For the 2BPP, $K = 10^{11}$ yields the best results, while both larger and smaller values of K lead to inferior outcomes. For the 2BPP-G, $K = 10^{10}$ provides the best results, followed by $K = 10^{12}$. It is noteworthy that the effect of varying K is minimal, suggesting that for classic instances, computing (safe) dual bounds with a precision greater than what commercial solvers allow for 10^9 is not critical. Nevertheless, the proposed numerically exact algorithm has considerable potential to handle real-world instances, some of which may be significantly influenced by precision. Finally, since 2BPP is more extensively studied and applied than 2BPP-G, we select $K = 10^{11}$ to optimize 2BPP performance.

Table EC.2 Impact of Varying the Scaling Factor K

K	2BPP					2BPP-G				
	#Opt	#LB	#UB	Gap	Time	#Opt	#LB	#UB	Gap	Time
10^{13}	494	7217	7223	0.20	49.27	488	7220	7232	0.40	98.10
10^{12}	494	7217	7223	0.20	51.58	489	7220	7231	0.38	92.35
10^{11}	495	7218	7223	0.18	49.26	488	7220	7232	0.40	99.94
10^{10}	494	7217	7223	0.20	53.67	489	7220	7231	0.38	91.02
10^{09}	493	7217	7224	0.21	57.88	488	7220	7232	0.40	98.92

Table EC.3 Contribution of the Different Components of NE-BPC to the 2BPP Results

Class		Main Components													
		NE-BPC		Initial bounds		Root node		Enumeration		Diving heuristic		Rounding heuristic		SR cuts	
		#Opt	Time	#Opt	Time	#Opt	Time	#Opt	Time	#Opt	Time	#Opt	Time	#Opt	Time
1	50	50	0.90	44	0.83	6	0.06	0	0.00	0	0.00	0	0.01	1	0.00
2	50	50	0.06	50	0.04	0	0.00	0	0.00	0	0.00	0	0.00	0	0.00
3	50	50	4.09	41	2.71	8	1.32	1	0.05	1	0.05	1	0.16	0	0.00
4	50	50	0.62	50	0.60	0	0.00	0	0.00	0	0.00	0	0.00	0	0.00
5	50	50	6.40	36	5.09	13	1.30	1	0.00	1	0.00	1	0.10	0	0.00
6	50	49	73.19	49	1.43	0	71.74	0	0.00	0	0.00	0	0.02	0	0.00
7	50	50	7.36	36	4.69	13	2.04	1	0.61	0	0.24	1	0.50	1	0.00
8	50	50	6.04	43	2.79	7	3.23	0	0.00	0	0.00	0	0.10	1	0.00
9	50	50	0.32	50	0.31	0	0.00	0	0.00	0	0.00	0	0.00	0	0.00
10	50	46	393.63	33	5.72	9	240.59	4	147.30	4	127.52	4	22.79	0	0.23
Overall	500	495	49.26	432	2.42	56	32.03	7	14.80	6	12.78	7	2.37	3	0.02

Table EC.4 Contribution of the Different Components of NE-BPC to the 2BPP-G Results

Class		#I		Main Components													
				NE-BPC		Initial bounds		Root node		Enumeration		Diving heuristic		Rounding heuristic		SR cuts	
#Opt	Time	#Opt	Time	#Opt	Time	#Opt	Time	#Opt	Time	#Opt	Time	#Opt	Time	#Opt	Time		
1	50	50	0.89	44	0.82	6	0.06	0	0.00	0	0.00	0	0.01	1	0.00		
2	50	50	0.06	50	0.04	0	0.00	0	0.00	0	0.00	0	0.00	0	0.00		
3	50	50	6.11	40	2.56	10	3.54	0	0.00	0	0.00	0	0.32	0	0.00		
4	50	49	72.41	49	0.71	0	71.68	0	0.00	0	0.00	0	0.04	0	0.00		
5	50	50	6.89	36	4.96	14	1.92	0	0.00	0	0.00	0	0.14	0	0.00		
6	50	48	145.53	48	2.03	0	143.48	0	0.00	0	0.00	0	0.05	0	0.00		
7	50	50	7.60	36	4.47	13	2.62	1	0.50	0	0.18	1	0.45	1	0.00		
8	50	50	4.99	43	2.66	7	2.31	0	0.00	0	0.00	0	0.12	1	0.00		
9	50	50	0.17	50	0.16	0	0.00	0	0.00	0	0.00	0	0.00	0	0.00		
10	50	41	754.69	33	5.64	8	613.22	0	135.81	0	73.59	0	18.65	1	0.09		
Overall	500	488	99.94	429	2.41	58	83.88	1	13.63	0	7.38	1	1.98	4	0.01		

EC.2.3. Analysis of the Contribution of the Main Components of NE-BPC to Overall Performance

Table EC.3 reports the contribution of the main components of NE-BPC to the 2BPP results. Most instances (432 of 495, 87%) are solved by the initial bounds, with an average computation time of only 2.42 seconds. The remaining 63 instances are solved at the root node (56) or during BPC enumeration (7). In addition, the diving heuristic, rounding heuristic, and lower bound improved by SR cuts solve 6, 7, and 3 instances, respectively. The contributions of these main components to the 2BPP-G results exhibit a similar pattern, as shown in Table EC.4. The initial bounds still yield the largest number of optimal solutions, followed by the root node. SR cuts remain effective, improving the lower bound and proving optimality for 4 instances, while the enumeration's contribution is minimal, solving only 1 instance.

The pie charts in Figures EC.2 and EC.3 present the distribution of computation time among the main algorithm components and processes. Each chart is based on the total algorithm runtime. For 2BPP and

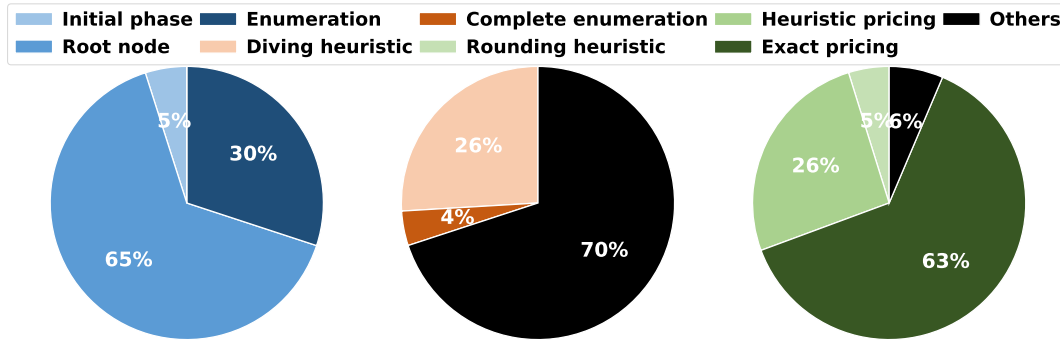


Figure EC.2 Computation Time Distribution of the Main Algorithm Components to 2BPP Results

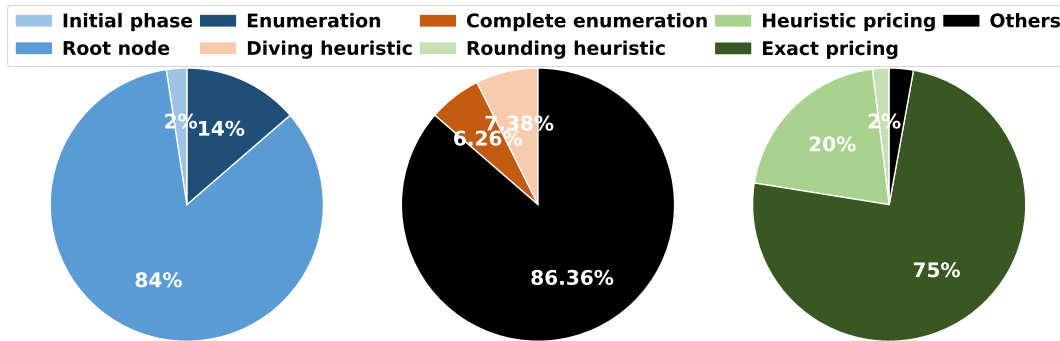


Figure EC.3 Computation Time Distribution of the Main Algorithm Components to 2BPP-G Results

2BPP-G, approximately 65% and 84% of the computation time, respectively, is spent solving the root node, which is the bottleneck of the proposed algorithm. During the enumeration phase, the diving heuristic consumes more time than complete enumeration; for 2BPP, it requires more than six times the computation time of complete enumeration. In the CRG process, the rounding heuristic takes very little computational resources while the pricing algorithm occupies almost all time, with exact pricing acting as the bottleneck by taking 63% and 75% of the total for 2BPP and 2BPP-G, respectively.

EC.2.4. Analysis of the Differences in the Results for 2BPP and 2BPP-G

To verify the close relationship between 2BPP and 2BPP-G, we analyze the results in Table EC.5, which show that the difference between the lower and upper bounds for the two problems on both the classic and new instances is minimal, with a maximum difference of no more than 0.2%. This indicates the close relationship between the two problems and demonstrates that 2BPP can serve as a very tight relaxation for 2BPP-G in most instances. Interestingly, the difference between the bounds for the two problems seems to increase as the number of items increases. We also observe that 2BPP-G appears more difficult to solve than 2BPP, requiring longer average computation times. This results in 2BPP-G exhibiting worse average lower-bound quality than 2BPP for the larger-scale new instances. Therefore, future research can explore how to leverage 2BPP as an effective relaxation to better solve large-scale 2BPP-G.

Table EC.5 Differences in the Results for 2BPP and 2BPP-G

Classic benchmark instances							New benchmark instances						
n	#LB		#UB		Time		n	#LB		#UB		Time	
	2BPP	2BPP-G	2BPP	2BPP-G	2BPP	2BPP-G		2BPP	2BPP-G	2BPP	2BPP-G	2BPP	2BPP-G
20	515	516	515	516	0.30	0.32	100	1281	1281	1286	1288	213.36	277.47
40	975	976	976	978	37.68	73.95	200	2493	2491	2500	2507	355.55	600.39
60	1442	1443	1444	1445	74.20	101.88	300	3637	3637	3651	3661	558.14	721.71
80	1959	1958	1960	1962	73.37	160.56	400	4900	4901	4919	4925	497.42	601.67
100	2327	2327	2328	2331	60.76	162.97	500	6016	6016	6044	6050	666.12	818.93
Overall	7218	7220	7223	7232	49.26	99.94	Overall	18327	18326	18400	18431	458.12	604.04

References

- Berkey JO, Wang PY (1987) Two-dimensional finite bin-packing algorithms. *Journal of the Operational Research Society* 38(5):423–429.
- Boschetti MA, Mingozzi A (2003) The two-dimensional finite bin packing problem. Part I: New lower bounds for the oriented case. *Quarterly Journal of the Belgian, French and Italian Operations Research Societies* 1(1):27–42.
- Carlier J, Clautiaux F, Moukrim A (2007) New reduction procedures and lower bounds for the two-dimensional bin packing problem with fixed orientation. *Computers & Operations Research* 34(8):2223–2250.
- Carlier J, Néron E (2007) Computing redundant resources for the resource constrained project scheduling problem. *European Journal of Operational Research* 176(3):1452–1463.
- Clautiaux F, Alves C, Valério de Carvalho J (2010) A survey of dual-feasible and superadditive functions. *Annals of Operations Research* 179(1):317–342.
- Cui YP, Yao Y, Zhang D (2018) Applying triple-block patterns in solving the two-dimensional bin packing problem. *Journal of the Operational Research Society* 69(3):402–415.
- Johnson DS (1973) *Near-optimal bin packing algorithms*. Ph.D. thesis, Massachusetts Institute of Technology.
- Martello S, Vigo D (1998) Exact solution of the two-dimensional finite bin packing problem. *Management Science* 44(3):388–399.
- Parreño F, Alvarez-Valdés R, Oliveira JF, Tamarit JM (2010) A hybrid GRASP/VND algorithm for two-and three-dimensional bin packing. *Annals of Operations Research* 179(1):203–220.
- Wang S, Baldacci R, Furini F, Wei L, Liu Q (2025a) Exact algorithms for two-dimensional knapsack problems: A unified framework with new benchmark results. *Available at SSRN 5322387*.
- Wang S, Baldacci R, Liu Q, Wei L (2025b) EATKG: An open-source efficient exact algorithm for the two-dimensional knapsack problem with guillotine constraints. *European Journal of Operational Research*.